



การพัฒนาระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์
DEVELOPMENT OF A DRIVER DROWSINESS DETECTION AND ALERT SYSTEM ON AN
EDGE-AI PLATFORM

นายสุรวี นันต์ธนะ
นายพงษ์สิทธิ์ เด็ดดวง
นางสาวเปรมฤดี สุขแสนดี

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
ภาควิชาวิศวกรรมอิเล็กทรอนิกส์และโทรคมนาคม คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี

พ.ศ. 2569

การพัฒนาระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์

นายสุรวิ นันต์ธนะ
นายพงษ์สิทธิ์ เด็ดดวง
นางสาวเปรมฤดี สุขแสนดี

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
ภาควิชาวิศวกรรมอิเล็กทรอนิกส์และโทรคมนาคม คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี

พ.ศ. 2569

DEVELOPMENT OF A DRIVER DROWSINESS DETECTION AND ALERT SYSTEM ON AN
EDGE-AI PLATFORM

MR.SURAWEE NANTHANA
MR.PONGSIT DEDDUANG
MS.PEMRUEDEE SUKSEANDEE

THIS PROJECT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE BACHELOR DEGREE OF ENGINEERING
DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION ENGINEERING
FACULTY OF ENGINEERING
RAJAMANGALA UNIVERSITY OF TECHNOLOGY THANYABURI
YEAR 2026

หัวข้อปริญญานิพนธ์	ระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์
นักศึกษา	นายสุรวิ นันต์ธนะ นายพงษ์สิทธิ์ เด็ดดวง นางสาวเปรมฤดี สุขแสนดี
อาจารย์ที่ปรึกษา	ผู้ช่วยศาสตราจารย์ ดร.ปราชญ์ อัครนรากุล

ภาควิชาวิศวกรรมอิเล็กทรอนิกส์และโทรคมนาคม คณะวิศวกรรมศาสตร์ มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี อนุมัติให้ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร วิศวกรรมศาสตรบัณฑิต

.....หัวหน้าภาควิชาฯ
(ผู้ช่วยศาสตราจารย์ ดร.ไพฑูรย์ รักเหลือ)

คณะกรรมการสอบปริญญานิพนธ์

.....ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ฉัตรชัย โชคชัย)

.....กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.จักรกฤษ อ่อนชื่นจิตร์)

.....กรรมการ
(อาจารย์ ดร.พิชัย สุขใสบุญ)

.....กรรมการและที่ปรึกษา
(ผู้ช่วยศาสตราจารย์ ดร.ปราชญ์ อัครนรากุล)

ลิขสิทธิ์ของภาควิชาวิศวกรรมอิเล็กทรอนิกส์และโทรคมนาคม คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเทคโนโลยีราชมงคลธัญบุรี

หัวปริญญาบัตร	ระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์		
นักศึกษา	นายสุรวิ	นันต์ธนะ	รหัส 116610400239-7
	นายพงษ์สิทธิ์	เด็ดดวง	รหัส 116610461226-0
	นางสาวเปรมฤดี	สุขแสนดี	รหัส 116610400198-5
อาจารย์ที่ปรึกษา	ผู้ช่วยศาสตราจารย์ ดร.ปราชญ์ อิศวนรากุล		
ปีการศึกษา	2569		

บทคัดย่อ

การหลับในระหว่างขับขี่ยานพาหนะเป็นสาเหตุสำคัญของอุบัติเหตุบนท้องถนนซึ่งสร้างความสูญเสียต่อชีวิต และทรัพย์สินอย่างมาก งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนา ระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ (Artificial Intelligence: AI) เพื่อเพิ่มความปลอดภัยในการขับขี่ โดยระบบถูกออกแบบให้ประมวลผลภาพจากกล้อง และวิเคราะห์พฤติกรรมของผู้ขับขี่แบบเรียลไทม์ด้วยโมเดลการเรียนรู้เชิงลำดับเวลา (Long Short-Term Memory: LSTM) ขั้นตอนการพัฒนาประกอบด้วยการเตรียม และแบ่งชุดข้อมูลแบบ Subject-wise การสกัดฟีเจอร์จากจุดใบหน้า เช่น อัตราส่วนการเปิดตา (Eye Aspect Ratio: EAR) และอัตราส่วนการอ้าปาก (Mouth Aspect Ratio: MAR) การสร้างลำดับเวลา (Sequence Generation) การเพิ่ม และปรับสมดุลข้อมูล (Augmentation และ Balancing) และการฝึกสอนโมเดลด้วยภาษาไพทอน (Python) ผลการทดสอบสามารถตรวจจับได้ 5 ส่วนหลัก ได้แก่ การตรวจจับดวงตา การตรวจจับการหาว การตรวจจับการขยับศีรษะ การตรวจจับการนำมือเข้าใกล้ใบหน้า และการตรวจจับทั้งหมดพร้อมกัน โดยเมื่อตรวจจับทั้งหมดพร้อมกัน ระบบให้ค่า Accuracy 91.77%, Precision 67.86%, Recall 69.09% และค่า F1-Score 68.47% แสดงให้เห็นว่าโมเดลสามารถตรวจจับอาการง่วงได้อย่างมีประสิทธิภาพ และมีความเหมาะสมต่อการพัฒนาเป็นต้นแบบของระบบแจ้งเตือนจริงในอนาคต

คำสำคัญ การพัฒนาระบบแจ้งเตือนการหลับในด้วยปัญญาประดิษฐ์ การประมวลผลภาพLSTM

กิตติกรรมประกาศ

ปริญญานิพนธ์เรื่องเล่มนี้สำเร็จลุล่วงไปได้ด้วยความกรุณา และการให้คำปรึกษาอย่างดียิ่งจาก ผู้ช่วยศาสตราจารย์ ดร.ปราชญ์ อัครนรากุล ซึ่งเป็นอาจารย์ที่ปรึกษาทั้งทางด้านวิชาการ และแนวทางการปฏิบัติมา การออกแบบกระบวนการทดลอง การพัฒนาโมเดลปัญญาประดิษฐ์ ตลอดจนแนวทางการจัดทำรายงานอย่างละเอียดรอบคอบ รวมถึงให้คำแนะนำ และกำลังใจมา โดยตลอด ตราบจนอาจารย์ภาควิชาวิศวกรรมอิเล็กทรอนิกส์และโทรคมนาคมทุกท่านที่ให้คำแนะนำ และให้ความช่วยเหลือในการแก้ไขข้อบกพร่องต่าง ๆ เพื่อให้ปริญญานิพนธ์ฉบับนี้มีความ สมบูรณ์ ซึ่งผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

เนื้อหาของปริญญานิพนธ์เล่มนี้จะไม่อาจสำเร็จลุล่วง และสมบูรณ์ได้ หากไม่ได้รับความอนุเคราะห์จากเพื่อนผู้ร่วมทำโครงงาน และคณาจารย์ที่ปรึกษาทุกท่าน ซึ่งได้กรุณาให้ความช่วยเหลือและคำแนะนำด้วยความเอื้ออาทรตลอดระยะเวลาการดำเนินงาน ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้

ข้าพเจ้าขอกราบขอบพระคุณบิดา มารดา และครอบครัว ที่ได้เป็นกำลังใจอันสำคัญยิ่ง คอยให้คำแนะนำ แนวคิด และการสนับสนุนในทุก ๆ ด้านมาโดยตลอด อันเป็นแรงผลักดันให้ข้าพเจ้าสามารถจัดทำปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงด้วยดี

คณะผู้จัดทำ

บทที่ 1

บทนำ

1.1 หลักการ และเหตุผล

การหลับในขณะขับขี่ยานพาหนะเป็นหนึ่งในสาเหตุสำคัญของอุบัติเหตุบนท้องถนน ซึ่งสร้างความสูญเสียทั้งต่อชีวิต และทรัพย์สินของประชาชนเป็นจำนวนมาก จากข้อมูลของกรมการขนส่งทางบกพบว่า อุบัติเหตุที่เกิดจากความง่วงหรือการหลับในมีแนวโน้มเพิ่มขึ้นทุกปี โดยเฉพาะในช่วงเวลากลางคืนหรือระยะทางไกล สาเหตุหลักมักมาจากความเหนื่อยล้า การพักผ่อนไม่เพียงพอ หรือปัญหาสุขภาพของผู้ขับขี่เอง

แม้ในปัจจุบันจะมีเทคโนโลยีช่วยเหลือผู้ขับขี่ (Driver Assistance Systems) เช่น ระบบเตือนออกนอกเลน หรือระบบควบคุมความเร็วอัตโนมัติ แต่ระบบเหล่านี้ยังไม่สามารถตรวจจับภาวะง่วงนอนหรือหลับในของผู้ขับขี่ได้อย่างแม่นยำ และแบบเรียลไทม์ ดังนั้นปัญญาประดิษฐ์ (AI) จึงมุ่งเน้นการพัฒนา ระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะโดยใช้ปัญญาประดิษฐ์ (AI) ซึ่งประมวลผลภาพจากกล้องตรวจจับใบหน้าผู้ขับขี่เพื่อวิเคราะห์พฤติกรรมที่บ่งชี้ถึงความง่วง เช่น การกะพริบตาถี่ การหาว หรือการเอียงศีรษะ ระบบจะประมวลผลข้อมูลแบบเรียลไทม์ และส่งสัญญาณแจ้งเตือนเมื่อพบว่าผู้ขับขี่มีแนวโน้มเข้าสู่ภาวะหลับใน

การพัฒนาระบบดังกล่าวจะช่วยลดความเสี่ยงจากอุบัติเหตุ เพิ่มความปลอดภัยในการเดินทาง และสามารถต่อยอดสู่การใช้งานในยานยนต์อัจฉริยะ (Smart Vehicle) ในอนาคตได้อีกด้วย

1.2 วัตถุประสงค์

- 1.2.1 ศึกษาและพัฒนาระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์
- 1.2.2 ศึกษาอัลกอริทึม การวิเคราะห์ท่าทางบุคคล และการใช้งานโมดูลปัญญาประดิษฐ์
- 1.2.3 ประยุกต์ใช้โมดูลปัญญาประดิษฐ์เพื่อการออกแบบระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะ

1.3 ขอบเขต

- 1.3.1 ออกแบบและพัฒนาระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ Eage-AI

1.3.2 การเรียนรู้จำอาการหลัก 4 รูปแบบ การกระพริบตา การสัปหงก การหาว การยกมือขึ้นมา บริเวณใบหน้า ไม่น้อยกว่า 100 ตัวอย่าง

1.3.3 ประเมินค่าทางสถิติของการแจ้งเตือนการหลับในได้ไม่น้อยกว่าร้อยละ 90

1.4 ผลที่คาดว่าจะได้รับ

1.4.1 ให้มีความรู้ความเข้าใจเกี่ยวกับเกี่ยวกับการรู้จำของปัญญาประดิษฐ์สำหรับการประยุกต์ใช้ในการออกแบบ และสร้างระบบแจ้งเตือนการหลับในด้วยปัญญาประดิษฐ์

1.4.2 ทำให้มีความรู้การใช้ความเข้าใจเกี่ยวกับการเขียนโปรแกรมด้วยภาษาไพทอน (Python)

1.4.3 ออกแบบ และพัฒนาอัลกอริทึม การรู้จำของปัญญาประดิษฐ์เพื่อนำมาประยุกต์ใช้กับระบบแจ้งเตือนการหลับใน

1.4.4 ทำให้วิเคราะห์สาเหตุ และแนวทางแก้ไขปัญหามาได้

1.4.5 สามารถนำผลการวิเคราะห์จากปัญญาประดิษฐ์ไปเปรียบเทียบกับผลการตรวจสอบที่ถูกต้อง

บทที่ 2

ทฤษฎี และงานวิจัยที่เกี่ยวข้อง

ในการทำปริญญานิพนธ์ครั้งนี้จำเป็นต้องมีความรู้ และความเข้าใจในเรื่อง ระบบแจ้งเตือน การ

หลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ การออกแบบปัญญาประดิษฐ์ และการประยุกต์ใช้งานการประเมินผล และความแม่นยำของปัญญาประดิษฐ์ การประมวลผลสัญญาณ ซึ่งทฤษฎีที่เกี่ยวข้องในหัวข้อต่าง ๆ ดังต่อไปนี้

2.1 ปัญหาการหลับในของผู้ขับขี่

การหลับในเป็นหนึ่งในสาเหตุสำคัญที่ทำให้เกิดอุบัติเหตุบนท้องถนน ซึ่งมีความเสี่ยงเกิดอุบัติเหตุได้สูงมาก เนื่องจากการหลับในทำให้ผู้ขับขี่สูญเสียการควบคุมรถ ทำให้เกิดอุบัติเหตุได้ง่าย การหลับในมักเกิดขึ้นขณะขับรถทางตรงยาว ๆ หรือรถติด ยิ่งถ้าเป็นช่วงวันหยุดในเทศกาลต่าง ๆ ที่การสัญจรบนถนนคับคั่งกว่าปกติ ทำให้การเคลื่อนตัวของรถช้าลง อาจส่งผลให้ผู้ขับขี่มักเกิดอาการง่วงนอน ไม่สดชื่น เดินทางในช่วงเวลาพลบค่ำหรือช่วงบ่าย ซึ่งเป็นช่วงเวลาที่จะเกิดอาการง่วงนอนได้ง่ายมากขึ้น อาจทำให้เกิดการหลับใน ได้ง่าย และอาจเกิดอุบัติเหตุได้ง่ายเช่นกัน

2.1.1 สาเหตุ และผลกระทบของการหลับใน

สาเหตุของการหลับในสามารถเกิดขึ้นได้จากหลายปัจจัยไม่ว่าจะเป็นการพักผ่อนที่ไม่เพียงพอ อ่อนเพลียเป็นเวลานาน ๆ การขับรถเป็นเวลานานโดยไม่พัก หรือเป็นโรคเกี่ยวกับการนอนหลับ เช่น โรคหยุดหายใจขณะหลับ (Sleep apnea) หรือโรคนอนไม่หลับ (Insomnia) อาจทำให้เกิดความอ่อนเพลีย และส่งผลให้เกิดการหลับในได้ นอกจากนี้ปัจจัยที่กล่าวมาการทานยาบางชนิด เช่น ยาแก้แพ้ ยาคลายกล้ามเนื้อ ยาแก้เมารถ หรือยานอนหลับ ยังสามารถทำให้รู้สึกง่วงนอนขณะขับรถได้ ถึงแม้จะทานยานั้น ๆ ก่อนนอนก็ตาม ยาบางชนิดสามารถออกฤทธิ์ได้ยาวนาน ซึ่งเพิ่มความเสี่ยงในการหลับใน หากจำเป็นต้องทานยาเหล่านี้ ควรปรึกษาแพทย์หรือเภสัชกรเกี่ยวกับผลข้างเคียง และ การขับชื้ออย่างปลอดภัยหลังจากทานยา

2.1.2 ผลกระทบจากการหลับใน

ผลกระทบจากการหลับในขณะขับรถนั้นรุนแรงกว่าที่หลายคนคิด หากผู้ขับขี่หลับในเพียงไม่กี่วินาที รถอาจเคลื่อนไปในทิศทางที่ไม่ควรไป ทำให้เกิดการชนหรืออุบัติเหตุอื่น ๆ ที่รุนแรงได้ ความเสียหายที่เกิดขึ้นไม่เพียงแต่ทำให้รถยนต์ได้รับความเสียหาย แต่ยังอาจส่งผลต่อชีวิต และทรัพย์สินของผู้อื่นได้ด้วย เพราะฉะนั้นนอกจากการพักผ่อนอย่างเพียงพอเพื่อป้องกันสาเหตุการหลับในแล้ว เพื่อป้องกันความเสี่ยงจากความเสียหายที่เกิดขึ้น จึงต้องมีการเตรียมตัวให้พร้อมก่อนขับรถ

2.1.3 วิธีป้องกันการหลับใน

1) การเตรียมตัวก่อนการเดินทาง

หากคุณรู้ว่าต้องออกเดินทาง และขับรถไกล ๆ การเตรียมตัวก่อนออกเดินทางเป็นเรื่องสำคัญ คุณต้องแน่ใจว่าคุณได้รับการพักผ่อนอย่างเพียงพอ โดยเฉพาะหากคุณเป็นคนขับ นอกจากนี้ควรวางแผนการเดินทางให้มีช่วงเวลาหยุดพักระหว่างทางก็เป็นเรื่องจำเป็น

2) การปรับพฤติกรรมระหว่างขับรถ

หากคุณรู้สึกง่วงนอนขณะขับรถควรรหาที่จอดพักทันที และพักผ่อนสักครู่ หลีกเลี่ยงการขับรถต่อไปในสภาพที่ร่างกายไม่พร้อม นอกจากนี้ยังมีวิธีที่จะช่วยให้คุณตื่นตัวระหว่างขับรถได้ คือ

- หยุดพักระหว่างการเดินทาง หยุดพักทุก ๆ 2 ชั่วโมง หรือหลังจากขับรถไป 100-150 กิโลเมตร
- ดื่มเครื่องดื่มที่มีคาเฟอีน การดื่มกาแฟหรือชาเขียวที่มีคาเฟอีนสามารถช่วยให้คุณตื่นตัวได้ชั่วคราว แต่ไม่ควรดื่มมากเกินไป
- เปิดหน้าต่างรับลมเย็น อากาศเย็นสามารถกระตุ้นประสาท และทำให้คุณรู้สึกสดชื่น ควรเปิดหน้าต่างรับลมเย็นหรือปรับแอร์ในรถให้เย็นขึ้น
- ฟังเพลงจังหวะเร็ว การฟังเพลงจังหวะเร็วที่คุณชื่นชอบจะช่วยกระตุ้นสมอง และทำให้คุณตื่นตัวมากขึ้น หรือจะร้องตามไปด้วยก็อาจช่วยทำให้คุณตื่นตัวมากขึ้น
- เคี้ยวหมากฝรั่งหรือขนมขบเคี้ยว การเคี้ยวหมากฝรั่งช่วยกระตุ้นประสาทสัมผัสและทำให้คุณรู้สึกตื่นตัว หรือการรับประทานขนมขบเคี้ยวระหว่างขับรถก็ได้เหมือนกัน
- พูดคุยกับผู้ร่วมเดินทาง เพื่อช่วยให้คุณไม่รู้สึกเบื่อหรือเหนื่อยล้าเกินไป

3) การดูแลสุขภาพเพื่อป้องกันการหลับใน

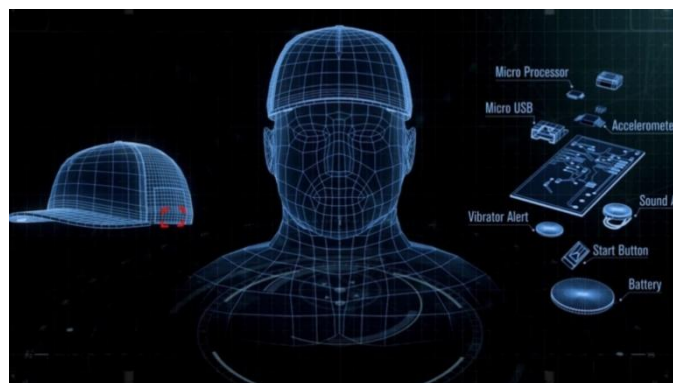
- การนอนหลับที่เพียงพอ การนอนหลับที่เพียงพอเป็นปัจจัยสำคัญในการป้องกันการหลับใน ควรนอนหลับอย่างน้อย 7-8 ชั่วโมงต่อคืน
- การดูแลสุขภาพจิตใจ สุขภาพจิตมีผลต่อการตื่นตัวขณะขับรถ หากคุณรู้สึกเครียดหรือมีปัญหาทางจิตใจ ควรหาวิธีผ่อนคลายก่อนการเดินทาง

2.2 เทคโนโลยีที่ใช้ในการตรวจจับการหลับใน

รวมอุปกรณ์อินเทอร์เน็ตในทุกสิ่ง (Internet of Things: IoT) อัจฉริยะป้องกันการหลับใน ช่วยชีวิตจากอุบัติเหตุ ในช่วงเทศกาลหยุดยาวเริ่มมีการออกเดินทางท่องเที่ยวกันมากขึ้น ซึ่งหากเป็นการเดินทางภายในประเทศ คนส่วนใหญ่มักใช้รถยนต์ส่วนตัวกันเป็นหลัก จึงทำให้เกิดเทคโนโลยี Wearable ที่สามารถช่วยให้การเดินทางปลอดภัยมากขึ้นดังนี้

2.2.1 หมวกอัจฉริยะป้องกันการหลับใน

หมวกดังกล่าวเป็นผลงานของฟอร์ด มอเตอร์ (Ford Motor) ที่ฉลองครบรอบ 60 ปีของการเปิดตัวรถบรรทุกในประเทศบราซิล ซึ่งฟอร์ดต้องการให้หมวกอัจฉริยะช่วยปกป้องชีวิตคนขับรถ โดยคอยเตือนให้มีสติเวลาที่คนขับรถเกิดง่วงขึ้นมา เพื่อช่วยรักษาชีวิตของทั้งคนขับ และคนใช้รถใช้ถนนอื่นๆ ให้ปลอดภัยมากขึ้นภายในหมวกจะมีทั้งไจโรสโคป (gyroscope) และแอ็กเซเลอมิเตอร์ (accelerometer) เพื่อช่วยในการเก็บข้อมูลของศีรษะคนขับว่ามีอาการคล้ายคนหลับในหรือไม่ ทั้งนี้ ฟอร์ดกล่าวว่า อุปกรณ์ชิ้นนี้อัจฉริยะมากพอที่จะแยกอาการหมุนคอไปมาเพื่อคลายปวดเมื่อยกล้ามเนื้อ กับอาการง่วงเพราะหลับในออกจากกันได้ ดังรูปที่ 2.1



รูปที่ 2.1 แบบจำลองโครงสร้างหมวกอัจฉริยะสำหรับตรวจจับภาวะง่วงนอนของผู้ขับขี่ [2]

2.2.2 แวรวิโก (Wearvigo)

ขั้นตอนการทำงานของแวรวิโก (Wearvigo) คือ บริเวณปลายอุปกรณ์จะมีเซ็นเซอร์ติดอยู่ ซึ่งตำแหน่งของมันจะอยู่ใกล้ ๆ กับดวงตาของผู้สวมใส่พอดี และจะคอยวิเคราะห์ความถี่ตัวของผู้สวมใส่ไว้แบบละเอียด ทั้งจังหวะการกะพริบตาว่าใช้เวลาอย่างน้อยขนาดไหน และการเคลื่อนไหวร่างกาย จากนั้นข้อมูลทั้งหมดที่เก็บข้อมูลมาได้จะถูกส่งไปที่ที่แอปพลิเคชันบนมือถือ หากพบว่าผู้สวมใส่มีอาการง่วงนอน กะพริบตาช้าลง ตัวอุปกรณ์ที่ใส่อยู่จะส่งสัญญาณเตือนด้วยการสั่น เปิดเพลง หรือกะพริบไฟสี เพื่อให้ผู้สวมแวรวิโก (Wearvigo) สะดุ้งตื่นขึ้นมาทันที แต่ทั้งนี้ การแจ้งเตือนต่าง ๆ ขึ้นอยู่กับการตั้งค่าการใช้งานของผู้ใช้เองด้วย ดังรูปที่ 2.2



รูปที่ 2.2 อุปกรณ์แวรวิโก (Wearvigo) [2]

2.2.3 ลิวัลล์ (Livall)

หมวกสำหรับนักปั่นจักรยาน ลิวัลล์ (Livall) มาพร้อมกับ แอ็กเซเลอโรมิเตอร์ (Accelerometer) เพื่อตรวจจับความรุนแรงของการกระแทก และมีระบบขอความช่วยเหลือโดยอัตโนมัติในกรณีที่เกิดอุบัติเหตุกับผู้สวม โดยตัวหมวกจะเชื่อมต่อกับสมาร์ทโฟน และจะโทรไปตามหมายเลขโทรศัพท์ที่ผู้ใช้งานตั้งค่าเอาไว้ นอกจากนี้ในหมวกยังมีไมโครโฟนไว้ใช้พูดกับปลายทางเพื่อเล่าเหตุการณ์ต่าง ๆ อีกด้วย ดังรูปที่ 2.3



รูปที่ 2.3 ลิวัลล์ (Livall) หมวกสำหรับนักปั่น [2]

2.3 ปัญญาประดิษฐ์ (AI)

2.3.1 ความหมายของปัญญาประดิษฐ์

ปัญญาประดิษฐ์ (AI) หมายถึง เทคโนโลยีหรือระบบคอมพิวเตอร์ที่ถูกออกแบบให้สามารถเลียนแบบกระบวนการทางสติปัญญาของมนุษย์ เช่น การเรียนรู้ (Learning) การให้เหตุผล (Reasoning) การตัดสินใจ (Decision Making) และการแก้ปัญหา (Problem Solving) โดยมีวัตถุประสงค์เพื่อให้เครื่องจักรหรือโปรแกรมสามารถทำงานได้อย่างชาญฉลาดใกล้เคียงกับมนุษย์มากที่สุด AI ทำงานโดยอาศัยการประมวลผลข้อมูลจำนวนมากผ่านอัลกอริทึมทางคณิตศาสตร์ และแบบจำลองทางสถิติ เพื่อให้ระบบสามารถเรียนรู้รูปแบบของข้อมูล และพัฒนาความสามารถในการตัดสินใจอัตโนมัติได้ นอกจากนี้ AI ยังมีบทบาทสำคัญในการช่วยมนุษย์วิเคราะห์ข้อมูลเชิงลึก (Deep Data Analytics) ทำงานซ้ำ ๆ ได้อย่างรวดเร็ว และแม่นยำ รวมถึงลดข้อผิดพลาดจากมนุษย์ (Human Error)

แม้ปัญญาประดิษฐ์จะมีศักยภาพสูงในการประมวลผล และวิเคราะห์ข้อมูล แต่ยังมีข้อจำกัดในการรับรู้ และตีความข้อมูลเชิงประสาทสัมผัส เช่น การสัมผัส กลิ่น หรือการเข้าใจอารมณ์ของมนุษย์ ซึ่งยังคงเป็นประเด็นที่ต้องมีการพัฒนาอย่างต่อเนื่องในงานวิจัยด้าน AI

2.3.2 หลักการทำงานของปัญญาประดิษฐ์ (AI)

ระบบ AI ทำงานโดยการรับข้อมูล วิเคราะห์ข้อมูล และประมวลผล เพื่อให้ได้ผลตอบกลับมา ไม่ว่าจะเป็นผ่านการใช้คำพูด ข้อความ หรือการกระทำต่าง ๆ ผลที่ตอบกลับมาก็อยู่ที่ว่าเราต้องการให้ตอบกลับมาเป็นอย่างไร และนำผลลัพธ์นั้นมาใช้ประโยชน์ให้ตรงกับจุดประสงค์ของเรา

อีกทั้งยังสามารถใช้รูปแบบการทำงานนี้เพื่อคาดการณ์สถานการณ์ที่จะเกิดขึ้นต่อไปในอนาคตได้ เช่น แชท บอท (Chatbot) ที่ตอบข้อความอัตโนมัติได้เหมือนกับคน หรือความสามารถในการจดจำ ภาพ ซึ่งการทำงานของระบบทั้งหมดนั้นต้องถูกเขียนโปรแกรมขึ้นมา โดยการเขียนโปรแกรมของ AI นั้นจะเน้นไปที่ทักษะการรับรู้ต่าง ๆ ดังนี้

1) การเรียนรู้ (Learning) โดยจะเน้นไปที่การรับข้อมูล และสร้างกฎสำหรับการเปลี่ยนเป็นข้อมูลที่น่าไปใช้ได้จริง ซึ่งกฎนั้นเรียกว่า อัลกอริทึม (Algorithms) คือ กระบวนการแก้ปัญหาที่อธิบายเป็นขั้นตอนไว้อย่างชัดเจน

2) การใช้เหตุผล (Reasoning) เน้นการตัดสินใจเลือกอัลกอริทึมที่เหมาะสมเพื่อให้ได้ผลลัพธ์ที่ต้องการ

3) การแก้ไขข้อผิดพลาด (Self-correction) ในส่วนนี้จะได้รับการออกแบบเพื่อปรับแต่งอัลกอริทึมให้วิเคราะห์ได้อย่างละเอียด เพื่อรับประกันว่าจะได้ผลลัพธ์ที่แม่นยำที่สุด

2.3.3 จุดประสงค์ของการใช้งานเทคโนโลยีปัญญาประดิษฐ์ (AI)

ระบบ AI มีความสามารถหลากหลายในการใช้งานกับธุรกิจ และอุตสาหกรรม ซึ่งจุดประสงค์หลักของการใช้เทคโนโลยี AI ก็เพื่อเพิ่มประสิทธิภาพในกระบวนการทำงาน ช่วยวิเคราะห์และประมวลผลข้อมูลจำนวนมากได้อย่างง่ายดาย และรวดเร็ว เพื่อดึงข้อมูลเชิงลึก (Insight) มาใช้กับธุรกิจ และอุตสาหกรรมได้อย่างแม่นยำนอกจากนี้ เทคโนโลยี AI ยังช่วยลดความผิดพลาดในการทำงานที่อาจเกิดขึ้นได้ รวมไปถึงใช้ค้นหาพฤติกรรมของผู้บริโภคที่ได้ขาดชุดข้อมูลที่มี เพื่อยกระดับให้ธุรกิจ หรืออุตสาหกรรมมีศักยภาพที่แข็งแกร่งในการแข่งขันให้ธุรกิจเติบโตได้ในระยะยาว

2.3.4 ประเภทของเทคโนโลยี AI ในปัจจุบัน

ประเภทของ AI สามารถแบ่งได้ 2 ประเภทใหญ่ คือ แบ่งตามความสามารถ และแบ่งตามฟังก์ชันการทำงาน โดยมีรายละเอียดดังนี้

1) แบ่งตามความสามารถของปัญญาประดิษฐ์ (AI)

ประเภทแบ่งตามความสามารถเป็นการแบ่งแบบทั่วไป คือการแบ่งตามพัฒนาการของ AI โดยมีดังนี้

- ปัญญาประดิษฐ์แบบแคบ (Artificial Narrow Intelligence: ANI) หรือปัญญาประดิษฐ์แบบเบา (Weak AI)

- ปัญญาประดิษฐ์ทั่วไป (Artificial General Intelligence: AGI) หรือปัญญาประดิษฐ์แบบเข้ม (Strong AI)

- ปัญญาประดิษฐ์ระดับเหนือมนุษย์ (Artificial Super Intelligence: ASI)

2) แบ่งตามฟังก์ชันการทำงานของปัญญาประดิษฐ์ (AI)

ประเภทแบ่งตามฟังก์ชันการทำงานหรือแบ่งจากระบบการประมวลผลของ AI จะแยกย่อยได้ ดังนี้

- รีแอคทีฟ แมชชีนส์ (Reactive Machines) เป็นระบบ AI ที่ไม่สามารถเรียนรู้ได้ด้วยตนเอง มีความสามารถจำกัด ไม่มีหน่วยความจำเป็นของตัวเอง ไม่สามารถดึงข้อมูลเก่ามาพัฒนาในการตัดสินใจให้ดีขึ้นได้

- ลิมิเตด ทีออรี (Limited Theory) เป็นระบบ AI ที่สามารถเรียนรู้ด้วยตนเองได้ มีหน่วยความจำ แต่มีจำกัด สามารถดึงข้อมูลเก่ามาตัดสินใจ หรือแก้ปัญหาในอนาคตได้

- ทีโอรี ออฟ ไมนด์ (Theory of Mind) การทำงานของ AI ประเภทนี้ คือ สามารถเข้าใจอารมณ์ ความรู้สึก วัฒนธรรม และความเชื่อต่าง ๆ ของมนุษย์ได้

- เซลฟ์อะแวร์เนส (Self-awareness) เป็นระบบ AI ขั้นสูงสุด สามารถมีอารมณ์ ความรู้สึก ความเชื่อ และมีความต้องการเป็นของตัวเอง รวมถึงสามารถคิดตัดสินใจ เลือกระหว่างสิ่งที่ต่าง ๆ ได้ด้วยตนเองทั้งหมด

2.3.5 การใช้งาน AI เพื่อดูแลรักษาความปลอดภัย

ปัจจุบันการทำธุรกิจมีขั้นตอนต่าง ๆ มากขึ้น และมีความซับซ้อนตามมาด้วย การป้องกันจึงต้องใช้เทคโนโลยี AI เข้ามาช่วย จึงได้มีระบบซอฟต์แวร์แอนตี้ไวรัสที่มีเทคโนโลยี AI อยู่เบื้องหลังการทำงาน ด้วยระบบ การเรียนรู้ของเครื่อง (Machine Learning: ML) จึงทำให้ระบบขององค์กรมีความปลอดภัยมากขึ้น สำหรับเรื่องรักษาความปลอดภัย มี AI ตรวจจับใบหน้าเพื่อคัดกรองบุคลากรในองค์กร หรือบุคคลแปลกหน้าได้ และการรักษาความปลอดภัย เช่น การตรวจจับและแจ้งเตือน หากพนักงานใส่อุปกรณ์รักษาความปลอดภัย (Personal Protection Equipment: PPE) ไม่ครบหรือไม่เหมาะสมในการทำงานในแต่ละพื้นที่ของพนักงาน ดังรูปที่ 2.4



รูปที่ 2.4 การดูแลรักษาความปลอดภัย [4]

2.3.6 ผลกระทบของ AI ต่อเศรษฐกิจ

สำหรับธุรกิจด้านนี้ ระบบ AI เข้ามามีบทบาทได้อย่างหลายอย่าง เช่น ช่วยพัฒนากระบวนการขาย การดูแล และให้บริการลูกค้า รวมถึงการวิเคราะห์ข้อมูลต่าง ๆ เพื่อดูความต้องการของลูกค้าในแต่ละกลุ่มได้ รวมถึงการเก็บข้อมูลประวัติการใช้งานเว็บไซต์ การวิเคราะห์ข้อมูล เพื่อนำมาใช้ในการโฆษณาทางออนไลน์ และนำมาปรับปรุงใช้กับระบบการขาย และการบริการให้ดียิ่งขึ้นนอกจากนี้ ระบบ AI ยังสามารถวิเคราะห์พฤติกรรมลูกค้า และวิเคราะห์การตลาดจากข้อมูลเชิงลึก ซึ่งจะช่วยให้ธุรกิจสามารถประเมินความสำเร็จ และทำให้เกิดการพัฒนาธุรกิจให้มีประสิทธิภาพมากขึ้นได้ในอนาคต อีกทั้ง AI ยังเข้ามาช่วยยกระดับการสร้างประสบการณ์ให้กับลูกค้าได้เป็นอย่างดี ซึ่งช่วยในการเพิ่มยอดขายได้

2.4 การประมวลผลภาพในการตรวจจับใบหน้า

2.4.1 ความหมายของการประมวลผลภาพ (Image Processing)

การประมวลผลภาพ (Image Processing) คือกระบวนการนำข้อมูลภาพที่ได้จากกล้องหรือเลนส์มาวิเคราะห์ และปรับปรุงคุณภาพของภาพให้เหมาะสม โดยเมื่อเลนส์ทำการจับภาพแล้ว โปรแกรมประมวลผลภาพจะทำการวิเคราะห์ภาพทันที เพื่อหาจุดโฟกัสที่เหมาะสม ปรับสมดุลของแสง หรือตัดเฉยแสงเพื่อให้ภาพมีความคมชัดมากยิ่งขึ้น ปัจจุบันเทคโนโลยีการประมวลผลภาพถูกนำมาใช้ในโปรแกรมคอมพิวเตอร์ และแอปพลิเคชันต่าง ๆ สำหรับการตกแต่งหรือปรับแต่งภาพที่ใช้กันอย่างแพร่หลายในชีวิตประจำวัน

2.4.2 หลักการทำงานของ การประมวลผลภาพ (Image Processing)

เริ่มต้นจากการนำภาพที่ต้องการเข้าสู่ระบบ จากนั้นซอฟต์แวร์ (Software) จะทำการวิเคราะห์ และแปลงภาพให้อยู่ในรูปแบบของข้อมูลดิจิทัล เพื่อให้คอมพิวเตอร์สามารถประมวลผล

ได้อย่างมีประสิทธิภาพ หลังจากนั้นจะมีการดึงข้อมูลสำคัญจากภาพ เช่น ขนาด รูปร่าง หรือ ลักษณะเฉพาะ เพื่อใช้ในการวิเคราะห์ และจัดทำเป็นระบบต่อไป เช่น การตรวจจับวัตถุ การจดจำ ใบหน้า หรือการจำแนกประเภทของภาพ

2.4.3 การปรับปรุงคุณภาพของภาพ (Image Enhancement)

เป็นกระบวนการในการจัดการการประมวลผลภาพของตัวเลข เพื่อสร้างภาพ รายละเอียดสูง หรือปรับโทนแสงของภาพ โดยนำภาพที่ถ่ายได้ ไปทำการเปรียบเทียบกับข้อมูล และรายละเอียดอื่น ๆ ของภาพชุดก่อน

2.4.4 การกำจัดสัญญาณอื่น ๆ ที่รบกวนภาพ (Image Noise Reduction)

เป็นการนำรูปไปกรองสัญญาณ ก่อนที่จะได้ภาพที่ต้องการออกมา โดยภาพที่กรองสัญญาณออกมานั้น จะมีคุณสมบัติแตกต่างจากภาพที่ใส่เข้าไปในระบบ โดยจุดประสงค์ของการกรองภาพนั้น คือการกำจัดทอนหรือเน้นคุณสมบัติบางอย่างของรูป เพื่อให้ได้รูปที่มีคุณลักษณะ ตามที่เราต้องการ

2.4.5 การสร้างภาพใหม่ด้วยการซ้อนทับภาพ (Image Composition)

เป็นการประมวลผลภาพ (Image Processing) ที่นำข้อมูลของภาพ ตั้งแต่สองภาพ ขึ้นไปมาซ้อนทับกัน เพื่อให้เกิดรูปใหม่ที่มีรายละเอียดของภาพ ที่สมบูรณ์มากยิ่งขึ้นเพื่อให้ได้ภาพ ใหม่ที่ได้นั้น มีรายละเอียดของข้อมูลที่มากเพียงพอต่อการนำไปใช้งานต่อไป

2.4.6 การคืนสภาพเดิมของภาพ (Image Restoration)

เป็นการทำให้รูปภาพที่ใส่เข้าไปในระบบนั้น คืนสู่สภาพเดิมหรืออาจจะมีการปรับปรุง รายละเอียดของภาพเพิ่มเติม เพื่อให้สามารถมองเห็นได้ชัดเจนมากยิ่งขึ้น

2.4.7 การแบ่งส่วนภาพ (Image Segmentation)

คือการตัดบางส่วนของภาพที่เราต้องการออกมาจากภาพที่เราใส่เข้าไปในระบบ ซึ่ง โดยส่วนใหญ่แล้วจะถูกนำไปใช้ในทางการแพทย์เป็นสำคัญ ซึ่งตามปกติแล้ว ภาพถ่ายที่ได้นั้น มักจะมีภาพติดของอวัยวะข้างเคียง หรืออาจจะมีสัญญาณรบกวนในขณะที่ถ่ายภาพติดมาด้วย เพราะฉะนั้น เราจึงต้องทำการตัดภาพอวัยวะที่ต้องการวิเคราะห์ออกมาอีกทีหนึ่ง

2.4.8 การตรวจจับขอบภาพ (Edge Detection)

เป็นกระบวนการหนึ่งในการประมวลผลภาพ (Image Processing) ที่ใช้สำหรับระบุ ขอบเขตหรือเส้นแบ่งของวัตถุภายในภาพ โดยอาศัยหลักการวิเคราะห์ความแตกต่างของค่าความ เข้มสีระหว่างพิกเซลข้างเคียง บริเวณที่มีการเปลี่ยนแปลงของความเข้มสีอย่างเด่นชัดจะถูกระบุว่าเป็นขอบภาพ จากนั้นระบบจะนำข้อมูลที่ได้ไปวิเคราะห์เพิ่มเติมเพื่อระบุรูปร่าง ขนาด หรือตำแหน่ง

ของวัตถุในภาพ ซึ่งขั้นตอนนี้มีความสำคัญอย่างมากในการประยุกต์ใช้งานด้านการรู้จำวัตถุ การตรวจจับใบหน้า และการประเมินคุณภาพของภาพ

2.4.9 การบีบอัดคุณภาพของภาพ (Image Compression)

เป็นกระบวนการลดขนาดของไฟล์ภาพให้เล็กลงโดยคงรายละเอียดสำคัญของภาพไว้มากที่สุด เพื่อให้สะดวกต่อการจัดเก็บ และส่งต่อข้อมูลในระบบสื่อสาร การบีบอัดภาพแบ่งออกเป็นสองประเภทหลัก ได้แก่

1) การบีบอัดแบบไม่สูญเสียรายละเอียดข้อมูล (Lossless Compression) เป็นการบีบอัดที่ยังคงค่าความสว่าง และข้อมูลของแต่ละพิกเซลไว้ครบถ้วนทุกประการ โดยอาศัยเทคนิคการจัดเก็บข้อมูลเชิงตัวเลขที่มีประสิทธิภาพ ทำให้สามารถคืนค่าภาพกลับสู่สภาพเดิมได้โดยไม่มี ความแตกต่างจากต้นฉบับ

2) การบีบอัดแบบสูญเสียรายละเอียดข้อมูล (Lossy Compression) เป็นการบีบอัดที่ลดขนาดไฟล์โดยยอมให้สูญเสียข้อมูลบางส่วน เช่น ค่าความสว่างหรือสีของพิกเซลบางตำแหน่ง ซึ่งอาจทำให้รายละเอียดบางส่วนของภาพเปลี่ยนแปลงไป วิธีนี้เหมาะกับการใช้งานทั่วไป เช่น การแสดงผลบนเว็บไซต์หรือสื่อออนไลน์ แต่ไม่เหมาะสำหรับงานที่ต้องใช้ภาพในการวิเคราะห์หรือจำแนกข้อมูลต่อไป เนื่องจากข้อมูลบางส่วนอาจสูญหายไปจากการบีบอัด

2.4.10 การสร้างภาพ 3 มิติ (3D Image Generation)

จากคุณสมบัติต่าง ๆ ของเทคโนโลยีการประมวลผลภาพ (Image Processing) สามารถนำไปประยุกต์ใช้ในการสร้างภาพสามมิติ (3D Image Generation) เพื่อใช้ในงานทางการแพทย์ได้อย่างมีประสิทธิภาพ บุคลากรทางการแพทย์สามารถนำภาพสามมิติที่ได้มาวิเคราะห์โรคหรือประเมินความผิดปกติของอวัยวะภายในได้อย่างละเอียด และแม่นยำมากขึ้น ปัจจุบันภาพสามมิติได้รับความนิยมอย่างแพร่หลาย เนื่องจากช่วยให้สามารถมองเห็นภาพรวม และรายละเอียดของอวัยวะในมุมมองต่าง ๆ ได้อย่างสมจริง โดยเฉพาะการวิเคราะห์ในส่วนของกระดูก ฟัน ขากรรไกร สมอง และหัวใจ ซึ่งเป็นข้อมูลสำคัญในการวินิจฉัย และวางแผนการรักษาทางการแพทย์ในยุคดิจิทัล

2.4.11 ประโยชน์ของการประมวลผลภาพ (Image processing)

เทคโนโลยีการประมวลผลภาพ (Image Processing) ได้รับการพัฒนาอย่างต่อเนื่อง และถูกนำมาประยุกต์ใช้อย่างกว้างขวางในชีวิตประจำวัน โดยเฉพาะในด้านความปลอดภัย การแพทย์ อุตสาหกรรม และระบบอัตโนมัติ ซึ่งช่วยเพิ่มประสิทธิภาพ ความแม่นยำ และ

ความสะดวกในการทำงาน เทคโนโลยีนี้ถือเป็นหนึ่งในวิทยาการที่สามารถนำไปใช้ประโยชน์ได้หลากหลาย ดังรายละเอียดต่อไปนี้

1) ประโยชน์ด้านกระบวนการผลิต

ในปัจจุบันเทคโนโลยีการประมวลผลภาพ (Image Processing) ถูกนำมาใช้ประโยชน์อย่างกว้างขวางในระบบการผลิตของโรงงานอุตสาหกรรม โดยสามารถทำงานร่วมกับเทคโนโลยีอื่น เช่น เทคโนโลยีตรวจจับความเคลื่อนไหว (Motion Detection) เพื่อใช้ในการตรวจสอบสภาพการทำงานภายในสายการผลิต ติดตามการปฏิบัติงานของพนักงานแบบเรียลไทม์ และช่วยควบคุมกระบวนการผลิตให้เป็นไปตามมาตรฐานที่กำหนด นอกจากนี้ระบบยังสามารถตรวจจับข้อผิดพลาดหรือความผิดปกติในกระบวนการผลิตได้อย่างรวดเร็ว ทำให้สามารถลดความสูญเสีย และเพิ่มประสิทธิภาพในการทำงานของโรงงานอุตสาหกรรมได้อย่างมีประสิทธิภาพ

2) ประโยชน์ด้านการจราจร

เทคโนโลยีการประมวลผลภาพ (Image Processing) สามารถนำมาประยุกต์ใช้ในการบริหารจัดการระบบจราจรได้อย่างมีประสิทธิภาพ โดยใช้ภาพจากกล้องวงจรปิดในการตรวจจับ และวิเคราะห์ข้อมูล เช่น การตรวจหาป้ายทะเบียนรถยนต์ (License Plate Recognition) เพื่อนำข้อมูลที่ได้มาแยกจัดเก็บเป็นหมวดหมู่ ได้แก่ หมายเลขทะเบียน ตัวอักษร จังหวัด ยี่ห้อ และสีของรถ เพื่อใช้ในการควบคุมหรือติดตามยานพาหนะในระบบ นอกจากนี้ยังสามารถนำมาใช้ในการนับจำนวนรถบนท้องถนน เพื่อตรวจสอบความหนาแน่นของการจราจร และวิเคราะห์สภาพการจราจรแบบเรียลไทม์ ซึ่งช่วยเพิ่มประสิทธิภาพในการวางแผนควบคุมสัญญาณไฟจราจร และลดปัญหาการจราจรติดขัดได้อย่างเหมาะสม

3) ประโยชน์ด้านการแพทย์

เทคโนโลยีการประมวลผลภาพ (Image Processing) ถูกนำมาใช้ในการแพทย์อย่างแพร่หลาย โดยเฉพาะในการทำงานร่วมกับเครื่องเอกซเรย์คลื่นแม่เหล็กไฟฟ้า (Magnetic Resonance Imaging: MRI) ซึ่งใช้สำหรับสร้างภาพอวัยวะภายในร่างกายในระนาบต่าง ๆ ได้อย่างละเอียด โดยเฉพาะในส่วนที่ไม่ใช่กระดูก ภาพที่ได้จากเครื่อง MRI สามารถให้ข้อมูลทั้งทางกายภาพ และทางเคมีของอวัยวะนั้น ๆ ได้อย่างแม่นยำ อีกทั้งผู้ป่วยไม่จำเป็นต้องเคลื่อนไหวระหว่างการตรวจ ทำให้สามารถวิเคราะห์ความผิดปกติภายในร่างกายได้อย่างปลอดภัย และมีประสิทธิภาพสูง

4) ประโยชน์ด้านการจัดระเบียบการจราจรในอาคาร

เทคโนโลยีการประมวลผลภาพ (Image Processing) สามารถนำมาประยุกต์ใช้
ในระบบการจัดการที่จอดรถภายในอาคารได้อย่างมีประสิทธิภาพ โดยมีการพัฒนาโปรแกรม
ตรวจสอบป้ายทะเบียนรถยนต์ที่ใช้เทคนิคการจดจำ และแยกตัวอักษรจากภาพถ่าย เพื่อ
เปรียบเทียบกับฐานข้อมูลที่จัดเก็บไว้ ระบบดังกล่าวสามารถตรวจสอบการเข้าหรือออกของ
ยานพาหนะได้แบบอัตโนมัติ และแสดงผลแบบเรียลไทม์ ซึ่งช่วยลดภาระงานของพนักงาน ลด
ความผิดพลาดในการบันทึกข้อมูล และเพิ่มความรวดเร็วในการจัดการพื้นที่จอดรถให้เป็นระบบ
มากยิ่งขึ้น

5) ประโยชน์ด้านการตรวจจับใบหน้า

การนำเทคโนโลยีการประมวลผลภาพ (Image Processing) มาประยุกต์ใช้ในการตรวจจับใบหน้า (Face Detection) ถือเป็นหนึ่งในประโยชน์ที่สำคัญ และได้รับความนิยมอย่างแพร่หลาย โดยระบบจะทำงานร่วมกับกล้องวงจรปิดคุณภาพสูง เพื่อจดจำ และวิเคราะห์ลักษณะใบหน้าของบุคคลที่อยู่ในภาพ จากนั้นจะเปรียบเทียบกับข้อมูลใบหน้าที่จัดเก็บไว้ในฐานข้อมูล เพื่อระบุตัวตนของบุคคลได้อย่างแม่นยำ เทคโนโลยีดังกล่าวถูกนำมาใช้ในระบบควบคุมการเข้าหรือออกของอาคาร การรักษาความปลอดภัยในสถานที่สำคัญ และระบบเฝ้าระวังอัจฉริยะ ซึ่งช่วยเพิ่มความปลอดภัย ลดการใช้บัตรผ่านทางกายภาพ และป้องกันการบุกรุกของบุคคลภายนอกได้อย่างมีประสิทธิภาพ ดังรูปที่ 2.5



รูปที่ 2.5 การตรวจจับใบหน้า [8]

2.5 การเขียนภาษาไพทอน (Python)

2.5.1 ความหมายของไพทอน (Python)

ภาษาไพทอน (Python) เป็นภาษาการเขียนโปรแกรมที่ได้รับความนิยมอย่างแพร่หลายในปัจจุบัน โดยถูกนำมาใช้ในการพัฒนาเว็บแอปพลิเคชัน การพัฒนาซอฟต์แวร์ (Software) งานด้านวิทยาศาสตร์ข้อมูล (Data Science) และ ML เนื่องจากไพทอน (Python) มีโครงสร้างภาษาที่เข้าใจง่าย มีประสิทธิภาพสูง และสามารถทำงานได้บนหลายแพลตฟอร์ม นอกจากนี้ยังเป็นซอฟต์แวร์แบบโอเพนซอร์ส (Open-Source Software: OSS) ที่สามารถดาวน์โหลด และใช้งานได้ฟรี รองรับการทำงานร่วมกับระบบคอมพิวเตอร์หลากหลายประเภท ช่วยเพิ่มความสะดวก และความเร็วในการพัฒนาโปรแกรม มีข้อดีคือ

- 1) นักพัฒนาสามารถอ่าน และทำความเข้าใจโปรแกรมไพทอน (Python) ได้ง่าย เนื่องจากมีไวยากรณ์พื้นฐานเหมือนภาษาอังกฤษ
- 2) ไพทอน (Python) ทำให้นักพัฒนาทำงานได้อย่างมีประสิทธิภาพมากขึ้น เนื่องจากสามารถเขียนโปรแกรมได้โดยใช้โค้ดน้อยลงเมื่อเปรียบเทียบกับภาษาอื่น ๆ อีกมากมาย
- 3) ไพทอน (Python) มีไลบรารี (Library) มาตรฐานขนาดใหญ่ที่มีโค้ดที่ใช้งานได้สำหรับเกือบทุกงาน ด้วยเหตุนี้ นักพัฒนาจึงไม่ต้องเขียนโค้ดขึ้นใหม่ทั้งหมด
- 4) นักพัฒนาสามารถใช้ไพทอน (Python) ร่วมกับภาษาการเขียนโปรแกรมยอดนิยมอื่น ๆ เช่น Java, C และ C++ ได้ง่าย
- 5) ชุมชนไพทอน (Python) ในปัจจุบันมีนักพัฒนาที่พร้อมให้การสนับสนุนหลายล้านคนทั่วโลก หากประสบปัญหา สามารถรับการสนับสนุนอย่างรวดเร็วได้จากชุมชน
- 6) มีแหล่งข้อมูลที่เป็นประโยชน์มากมายบนอินเทอร์เน็ต หากต้องการเรียนรู้ไพทอน (Python) สามารถค้นหาวิดีโอ บทแนะนำสอนการใช้งาน เอกสารประกอบ และคู่มือนักพัฒนาได้อย่างง่ายดาย
- 7) ไพทอน (Python) สามารถใช้งานได้บนระบบปฏิบัติการคอมพิวเตอร์ต่าง ๆ เช่น Windows, macOS, Linux และ Unix

2.5.2 การใช้งานไพทอน (Python)

ภาษาไพทอน (Python) มีกรณีการใช้งานหลายอย่างในการพัฒนาแอปพลิเคชัน ซึ่งรวมถึงตัวอย่างดังต่อไปนี้

1) การพัฒนาเว็บฝั่งเซิร์ฟเวอร์ (Server)

การพัฒนาเว็บฝั่งเซิร์ฟเวอร์ (Server) ประกอบด้วยฟังก์ชันแบ็คเอนด์ (Backend functions) ที่ซับซ้อนซึ่งเว็บไซต์ดำเนินการเพื่อแสดงข้อมูลต่อผู้ใช้ ตัวอย่างเช่น เว็บไซต์ต้องโต้ตอบกับฐานข้อมูล สื่อสารกับเว็บไซต์อื่น และปกป้องข้อมูลเมื่อส่งข้อมูลผ่านเครือข่ายไพทอน (Python) มีประโยชน์สำหรับการเขียนโค้ดฝั่งเซิร์ฟเวอร์ (Server) เนื่องจากมีไลบรารี (Library) จำนวนมากที่ประกอบด้วยโค้ดที่เขียนไว้ล่วงหน้าสำหรับฟังก์ชันแบ็คเอนด์ (Backend functions) ที่ซับซ้อน นักพัฒนายังใช้เฟรมเวิร์กไพทอน (Python Framework) ที่หลากหลายซึ่งมีเครื่องมือที่จำเป็นทั้งหมดเพื่อสร้างเว็บแอปพลิเคชันได้เร็วขึ้น และง่ายขึ้นอีกด้วย ตัวอย่างเช่น นักพัฒนาสามารถสร้างโครงสร้างเว็บแอปพลิเคชันได้ภายในไม่กี่วินาที เนื่องจากไม่จำเป็นต้องเขียนขึ้นใหม่ทั้งหมด จากนั้นนักพัฒนาสามารถทดสอบได้โดยใช้เครื่องมือทดสอบของเฟรมเวิร์ก (Framework) โดยไม่ต้องพึ่งพาเครื่องมือทดสอบภายนอก

2) ระบบอัตโนมัติด้วยไพทอนสคริปต์ (Python Script)

ภาษาการเขียนสคริปต์ (Script) คือภาษาการเขียนโปรแกรมที่ทำให้งานที่มนุษย์ทำตามปกติเป็นไปโดยอัตโนมัติ โปรแกรมเมอร์จึงใช้ไพทอนสคริปต์ (Python Script) อย่างแพร่หลายเพื่อทำให้งานประจำวันหลายอย่างดังต่อไปนี้เป็นไปโดยอัตโนมัติ

- การเปลี่ยนชื่อไฟล์จำนวนมากพร้อมกัน
- การแปลงไฟล์เป็นไฟล์ประเภทอื่น
- การลบคำที่ซ้ำกันในไฟล์ข้อความ
- การดำเนินการทางคณิตศาสตร์ขั้นพื้นฐาน
- การส่งข้อความอีเมล
- การดาวน์โหลดเนื้อหา
- การดำเนินการวิเคราะห์ขั้นพื้นฐาน
- การค้นหาข้อผิดพลาดในหลายไฟล์

3) วิทยาศาสตร์ข้อมูล (Data Scientist: DS) และการเรียนรู้ของเครื่อง (ML)

วิทยาศาสตร์ข้อมูล (DS) คือความรู้ที่มีคุณค่าจากข้อมูล และการเรียนรู้ของเครื่อง (ML) จะสอนคอมพิวเตอร์ให้เรียนรู้จากข้อมูลโดยอัตโนมัติ และทำนายได้อย่างแม่นยำ นักวิทยาศาสตร์ข้อมูลใช้ไพทอน (Python) สำหรับงานด้านวิทยาศาสตร์ข้อมูล (DS) ต่าง ๆ ดังต่อไปนี้

- การแก้ไข และลบข้อมูลที่ไม่ถูกต้อง ซึ่งเรียกว่าการทำความสะอาดข้อมูล
- การแยก และเลือกคุณสมบัติต่าง ๆ ของข้อมูล
- การระบุประเภทข้อมูล ซึ่งเป็นการเพิ่มชื่อที่มีความหมายสำหรับข้อมูล
- การค้นหาสถิติต่าง ๆ จากข้อมูล
- การแสดงข้อมูลด้วยภาพโดยใช้แผนภูมิ และกราฟ เช่น แผนภูมิเส้น กราฟแท่ง

ฮิสโทแกรม และแผนภูมิวงกลม

นักวิทยาศาสตร์ข้อมูลใช้ไลบรารี (Library) เพื่อฝึกฝนโมเดล ML และสร้างตัวจำแนกที่จำแนกประเภทข้อมูลได้อย่างแม่นยำ บุคคลในแวดวงต่าง ๆ ใช้ตัวจำแนกแบบไพทอน (Python) เพื่อทำงานด้านการจำแนกประเภท เช่น การจำแนกประเภทรูปภาพ ข้อความ และการรับส่งข้อมูลทางเครือข่าย การรู้จำเสียง และการจดจำใบหน้า นักวิทยาศาสตร์ข้อมูลยังใช้ไพทอน (Python) สำหรับ การเรียนรู้เชิงลึก (Deep Learning: DL) ซึ่งเป็นเทคนิค ML ขั้นสูง

4) การพัฒนาซอฟต์แวร์ (Software)

นักพัฒนาซอฟต์แวร์มักใช้ไพทอน (Python) สำหรับงานด้านการพัฒนา และการประยุกต์ใช้ซอฟต์แวร์ (Software) ต่าง ๆ ดังนี้

- การติดตามบั๊กในโค้ดของซอฟต์แวร์
- การสร้างซอฟต์แวร์โดยอัตโนมัติ
- การดูแลการจัดการโครงการด้วยซอฟต์แวร์
- การพัฒนาต้นแบบซอฟต์แวร์
- การพัฒนาแอปพลิเคชันบนเดสก์ท็อปโดยใช้ไลบรารีส่วนติดต่อผู้ใช้แบบ

กราฟิก (Graphical User Interface: GUI)

2.5.3 ระบบทดสอบซอฟต์แวร์อัตโนมัติ (Software Automated Testing)

การทดสอบซอฟต์แวร์เป็นกระบวนการตรวจสอบว่าผลลัพธ์จริงจากซอฟต์แวร์ตรงกับผลลัพธ์ที่คาดหวังหรือไม่ เพื่อให้แน่ใจว่าซอฟต์แวร์ (Software) ไม่มีข้อผิดพลาด

1) โดยนักพัฒนาใช้เฟรมเวิร์ก (Framework) ในการทดสอบหน่วยไพทอน Python เช่น Unit test, Robot และ PyUnit เพื่อทดสอบการทำงานที่เขียนขึ้น

2) ผู้ทดสอบซอฟต์แวร์ใช้ไพทอน (Python) เพื่อเขียนกรณีที่ใช้ในการทดสอบสำหรับสถานการณ์การทดสอบต่าง ๆ ตัวอย่างเช่น ผู้ทดสอบใช้เพื่อทดสอบอินเทอร์เฟซ (Interface) ผู้ใช้ของเว็บแอปพลิเคชัน ส่วนประกอบซอฟต์แวร์ (Software) หลายตัว และคุณสมบัติใหม่

นักพัฒนาสามารถใช้เครื่องมือหลายอย่างเพื่อเรียกใช้สคริปต์ (Script) ทดสอบได้โดยอัตโนมัติ โดยเครื่องมือเหล่านี้เรียกว่าเครื่องมือรวบรวม/ปรับใช้อัตโนมัติทั้งนี้ผู้ทดสอบ และนักพัฒนาซอฟต์แวร์ใช้เครื่องมือบูรณาการ (Continuous Integration: CI) และส่งมอบอย่างต่อเนื่อง (Continuous Deployment: CD) หรือ CI/CD เช่น Travis CI และ Jenkins เพื่อทำให้การทดสอบเป็นไปโดยอัตโนมัติ เครื่องมือ CI/CD จะเรียกใช้สคริปต์ทดสอบไพทอน (Python) โดยอัตโนมัติ และรายงานผลการทดสอบทุกครั้งที่นักพัฒนานำการเปลี่ยนแปลงโค้ดใหม่มาใช้

2.5.4 ประวัติความเป็นมาของไพทอน (Python)

Guido Van Rossum โปรแกรมเมอร์คอมพิวเตอร์ในเนเธอร์แลนด์เป็นผู้สร้างไพทอน (Python) โดยเขาเริ่มต้นในปี 1989 ที่ Centrum Wiskunde & Informatica ซึ่งแต่เดิมเป็นเพียงโครงการงานอดิเรกแก้เหงาในช่วงคริสต์มาส ทั้งนี้ชื่อของภาษาได้รับแรงบันดาลใจจากรายการโทรทัศน์ Monty Python's Flying Circus ของช่อง BBC TV เนื่องจาก Guido Van Rossum เป็นแฟนตัวยงของรายการดังกล่าว โดยประวัติของ Python เวอร์ชันต่าง ๆ แบ่งออกเป็นดังนี้

1) Guido Van Rossum เผยแพร่โค้ดไพทอน (Python) เวอร์ชันแรก (เวอร์ชัน 0.9.0) ในปี 1991 โดยมีคุณสมบัติต่าง ๆ ที่ดีอยู่แล้ว เช่น ประเภทข้อมูล และฟังก์ชันบางส่วนสำหรับการจัดการข้อผิดพลาด

2) Python 1.0 ได้รับการนำออกมาใช้ในปี 1994 พร้อมฟังก์ชันใหม่เพื่อประมวลผลรายการข้อมูลได้อย่างง่ายดาย เช่น Map, Filter และ Reduce

3) ในขณะที่ Python 2.0 ได้รับการนำออกมาใช้เมื่อวันที่ 16 ตุลาคม 2000 พร้อมคุณสมบัติใหม่ที่เป็นประโยชน์สำหรับโปรแกรมเมอร์ เช่น การรองรับอักขระ Unicode และวิธีที่สั้นกว่าในการวนลูปรายการ

4) เมื่อวันที่ 3 ธันวาคม 2008 ได้มีการนำ Python 3.0 ออกมาใช้ ซึ่งมีคุณสมบัติต่างๆ เช่น ฟังก์ชันการพิมพ์ และการสนับสนุนเพิ่มเติมสำหรับการแบ่งหมายเลข และการจัดการข้อผิดพลาด

2.5.5 คุณสมบัติของ Python

คุณสมบัติต่อไปนี้ทำให้ภาษาการเขียนโปรแกรม Python มีเอกลักษณ์ที่ไม่เหมือนใคร ได้แก่

1) ภาษาที่แปลผลแล้ว (Interpreted Language)

ไพทอน (Python) เป็นภาษาที่แปลผลแล้ว ซึ่งหมายความว่าสามารถเรียกใช้โค้ดที่ละบรรทัดได้โดยตรง หากมีข้อผิดพลาดในโค้ดโปรแกรม ก็จะหยุดทำงานทันที ดังนั้นโปรแกรมเมอร์จึงสามารถค้นหาข้อผิดพลาดในโค้ดได้อย่างรวดเร็ว

2) ภาษาที่ใช้งานง่าย

ไพทอน (Python) ใช้คำที่เหมือนในภาษาอังกฤษ ซึ่งแตกต่างจากภาษาการเขียนโปรแกรมอื่นๆ เนื่องจากไพทอน (Python) ไม่ใช้วงเล็บปีกกา แต่จะใช้การเยื้องแทน

3) ภาษาที่ระบุประเภทแบบไดนามิก (Dynamic Programming Language)

โปรแกรมเมอร์ไม่ต้องระบุประเภทตัวแปรเมื่อเขียนโค้ดเนื่องจากไพทอน (Python) จะกำหนดไว้ที่รันไทม์ (Runtime) ด้วยเหตุนี้ คุณจึงสามารถเขียนโปรแกรมไพทอน (Python) ได้รวดเร็วขึ้น

4) ภาษาโปรแกรมระดับสูง (High-Level Programming Language: HLPL)

ไพทอน (Python) มีความใกล้เคียงกับภาษามนุษย์มากกว่าภาษาการเขียนโปรแกรมอื่นๆ ดังนั้นโปรแกรมเมอร์จึงไม่ต้องกังวลกับฟังก์ชัน (Functions) การทำงานพื้นฐานต่างๆ เช่น สถาปัตยกรรม และการจัดการหน่วยความจำ (Memory Management)

5) ภาษาเชิงอ็อบเจกต์ (Object-Oriented Languages)

ไพทอน (Python) ถือว่าทุกสิ่งเป็นอ็อบเจกต์ (Object) แต่ก็ยังรองรับการเขียนโปรแกรมประเภทอื่นๆ ด้วย เช่น การเขียนโปรแกรมเชิงโครงสร้าง และเชิงฟังก์ชัน

2.5.6 ไลบรารีของไพทอน (Python Library)

ไลบรารี (Library) คือชุดของโค้ดที่ใช้อย่างน้อยหนึ่งครั้งโดยนักพัฒนาสามารถใช้ในโปรแกรมไพทอน (Python) เพื่อหลีกเลี่ยงการเขียนโค้ดซ้ำใหม่ทั้งหมด ตามค่าเริ่มต้นแล้วไพทอน (Python) จะมาพร้อมกับไลบรารีมาตรฐาน (Standard Library) ซึ่งมีฟังก์ชัน (Functions) ที่นำกลับมาใช้ใหม่ได้มากมาย นอกจากนี้ยังมีไลบรารีของไพทอน (Python Library) มากกว่า 137,000 รายการสำหรับการประยุกต์ใช้ต่าง ๆ รวมถึงการพัฒนาเว็บ DS และ ML โดยไลบรารีของไพทอน (Python Library) ยอดนิยมมีดังนี้

1) แมตพลอตลิบ (Matplotlib)

นักพัฒนาซอฟต์แวร์ใช้แมตพลอตลิบ (Matplotlib) เพื่อลงจุดข้อมูลในกราฟสองมิติ และสามมิติ (2D และ 3D) คุณภาพสูง ซึ่งมักจะใช้ในงานทางวิทยาศาสตร์ แมตพลอตลิบ (Matplotlib) ช่วยให้คุณสามารถแสดงข้อมูลเป็นภาพโดยแสดงผลในแผนภูมิต่าง ๆ เช่น แผนภูมิแท่ง และแผนภูมิเส้น คุณยังสามารถลงจุดแผนภูมิได้หลายรายการพร้อมกัน และกราฟสามารถใช้งานได้ในทุกแพลตฟอร์มอีกด้วย

2) แพนดัส (Pandas)

แพนดัส (Pandas) มีโครงสร้างข้อมูลที่ปรับให้เหมาะสม และยืดหยุ่น ซึ่งคุณสามารถใช้เพื่อจัดการชุดข้อมูลเวลา และข้อมูลที่มีโครงสร้าง เช่น ตาราง และอาร์เรย์ ตัวอย่างเช่น คุณสามารถใช้ Pandas เพื่ออ่าน เขียน ผสานรวม กรอง และจัดกลุ่มข้อมูลได้ หลายคนจึงใช้สำหรับงานด้านการวิเคราะห์ข้อมูล DS และ ML

3) นัมพาย (NumPy)

เป็นไลบรารี (Library) ยอดนิยมที่นักพัฒนาใช้เพื่อสร้าง และจัดการอาร์เรย์ (Array) จัดการรูปร่างเชิงตรรกะ และดำเนินการคำนวณพีชคณิตเชิงเส้นได้อย่างง่ายดาย โดย NumPy รองรับการทำงานร่วมกับภาษาต่าง ๆ มากมาย เช่น C และ C++

4) รีควีสต์ (Requests)

ใช้สำหรับส่ง และรับข้อมูลผ่านโปรโตคอล HTTP เหมาะสำหรับการพัฒนาเว็บแอปพลิเคชัน สามารถเพิ่มส่วนหัว พารามิเตอร์ หรือข้อมูลต่าง ๆ ในคำขอได้อย่างสะดวก ช่วยให้การสื่อสารระหว่างโปรแกรมกับเว็บเซิร์ฟเวอร์ (Web Server) เป็นไปอย่างง่าย และมีประสิทธิภาพ

5) โอเพนซีวี-ไพทอน (OpenCV-Python)

เป็นไลบรารี (Library) ที่นักพัฒนาใช้ในการประมวลผลรูปภาพสำหรับการประยุกต์ใช้เชิงคอมพิวเตอร์วิทัศน์ (Computer Vision) ซึ่งมีฟังก์ชันมากมายสำหรับการประมวลผลภาพ (Image Processing) เช่น การอ่าน และเขียนรูปภาพพร้อมกัน การสร้างภาพแวลูอม 3 มิติจาก 2 มิติ และการบันทึกภาพ และวิเคราะห์ภาพจากวิดีโอ

6) คีราส (Keras)

เป็นไลบรารี (Library) สำหรับสร้าง และพัฒนาโครงข่ายประสาทเทียมเชิงลึก (Deep Neural Network: DNN) บนภาษาไพทอน (Python) โดยมีโครงสร้างแบบโมดูลที่ยืดหยุ่น ใช้งานง่าย และรองรับการประมวลผลข้อมูล การแสดงผลข้อมูล รวมถึงการพัฒนาแอปพลิเคชันด้านปัญญาประดิษฐ์ และนวัตกรรมได้อย่างมีประสิทธิภาพ

2.5.7 เฟรมเวิร์กไพทอน (Python Framework)

คือชุดข้อมูลของแพ็คเกจ (Package) และโมดูล โดยโมดูลคือชุดของโค้ดที่เกี่ยวข้อง และแพ็คเกจคือชุดของโมดูล ทั้งนี้ นักพัฒนาสามารถใช้เฟรมเวิร์กไพทอน (Python Framework) เพื่อสร้างแอปพลิเคชันไพทอน (Python) ได้รวดเร็วขึ้น เนื่องจากไม่ต้องกังวลกับรายละเอียดระดับต่ำต่าง ๆ เช่น วิธีการที่การสื่อสารเกิดขึ้นในเว็บแอปพลิเคชัน หรือวิธีการที่ไพทอน (Python) จะทำให้โปรแกรมเร็วขึ้น โดยเฟรมเวิร์กไพทอน (Python Framework) มี 2 ประเภท ได้แก่ เฟรมเวิร์กแบบฟูลสแต็ก (Full-stack framework) มีเกือบทุกสิ่งที่จำเป็นสำหรับการสร้างแอปพลิเคชันขนาดใหญ่ และไมโครเฟรมเวิร์ก (Microframework) เป็นเฟรมเวิร์ก (Framework) พื้นฐานที่มีฟังก์ชันการทำงานน้อยที่สุดสำหรับการสร้างแอปพลิเคชันไพทอน (Python) อย่างง่าย นอกจากนี้ยังมีส่วนขยายหากแอปพลิเคชันต้องการฟังก์ชันที่ซับซ้อนยิ่งขึ้นอีก นักพัฒนาสามารถใช้เฟรมเวิร์กไพทอน (Python Framework) หลายแบบเพื่อให้การพัฒนามีประสิทธิภาพ ซึ่งรวมถึงเฟรมเวิร์ก (Framework) ต่าง ๆ ดังต่อไปนี้

1) จังโก (Django)

เป็นเฟรมเวิร์กแบบฟูลสแต็ก (Full-stack framework) ที่ใช้กันอย่างแพร่หลายมากที่สุดสำหรับการพัฒนาเว็บแอปพลิเคชันขนาดใหญ่ ซึ่งมีคุณสมบัติที่มีประโยชน์หลายอย่าง ซึ่งรวมถึงเว็บเซิร์ฟเวอร์ (Web Server) สำหรับการพัฒนา และทดสอบ โปรแกรมเทมเพลตสำหรับสร้างฟรอนต์เอนด์ (Front-end) ของเว็บไซต์ และกลไกต่าง ๆ ด้านความปลอดภัย

2) แพลสก์ (Flask)

เป็นไมโครเฟรมเวิร์ก (Microframework) สำหรับการพัฒนาเว็บแอปพลิเคชันขนาดเล็ก โดยคุณสมบัติของแพลสก์ (Flask) ได้แก่ การสนับสนุนที่แข็งแกร่งจากชุมชน เอกสารประกอบที่ให้อยู่ละเอียดครบถ้วน โปรแกรมเทมเพลต การทดสอบหน่วย และเว็บเซิร์ฟเวอร์ (Web Server) ในตัว นอกจากนี้ยังมีส่วนขยายสำหรับการสนับสนุนการตรวจสอบความถูกต้อง และความปลอดภัยของเว็บอีกด้วย

3) เทอร์โบเกียร์ส (TurboGears)

เป็นเฟรมเวิร์ก (Framework) ที่ออกแบบมาเพื่อสร้างเว็บแอปพลิเคชันได้เร็ว และง่ายขึ้น และคุณสมบัติยอดนิยมบางส่วนของ เทอร์โบเกียร์ส (TurboGears) ได้แก่

- โครงสร้างตารางฐานข้อมูลเฉพาะ
- เครื่องมือสำหรับสร้าง และจัดการโครงการ
- โปรแกรมเทมเพลตเพื่อสร้างฐานข้อมูล
- โปรแกรมเทมเพลตเพื่อสร้างฟรอนต์เอนด์ (Front-end)
- กลไกในการดูแลความปลอดภัยของเว็บ

4) อะพาชี เอ็มเอ็กซ์เน็ต (Apache MXNet)

เป็นเฟรมเวิร์กดีปเลิร์นนิง (Deep Learning Frameworks) ที่รวดเร็ว ยืดหยุ่น และปรับขนาดได้ ซึ่งนักพัฒนาใช้เพื่อสร้างต้นแบบการวิจัย และแอปพลิเคชัน DL ซึ่งรองรับภาษาการเขียนโปรแกรมหลายภาษา รวมถึง Java, C++, R และ Perl โดยมีชุดเครื่องมือ และไลบรารี (Library) ที่หลากหลายเพื่อรองรับการพัฒนา ตัวอย่างเช่น คุณสามารถค้นหาหนังสือ ML เชิงโต้ตอบ, ชุดเครื่องมือคอมพิวเตอร์วิทัศน์ (Computer Vision) และโมเดล DL สำหรับการประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) ซึ่งประมวลผลภาษาธรรมชาติต่าง ๆ เช่น ข้อความ และคำพูด

5) ไพทอร์ช (PyTorch)

เป็นเฟรมเวิร์ก (Framework) สำหรับ ML ที่สร้างขึ้นบนไลบรารี (Library) ทอร์ช (Torch) ซึ่งเป็นไลบรารี ML แบบโอเพนซอร์ส (Open Source) อีกแบบหนึ่ง โดยนักพัฒนาซอฟต์แวร์ใช้ PyTorch สำหรับแอปพลิเคชันต่าง ๆ เช่น NLP, วิทยาการหุ่นยนต์ (Robotics) และคอมพิวเตอร์วิทัศน์ (Computer Vision) โดยค้นหาข้อมูลที่มีความหมายในรูปภาพ และวิดีโอ พวกเขาใช้ PyTorch เพื่อเรียกใช้แอปพลิเคชันเหล่านั้นในหน่วยประมวลผลกลาง (Central

Processing Unit: CPU) และหน่วยประมวลผลกราฟิก (Graphics Processing Unit: GPU) อีกด้วย

2.5.8 สภาพแวดล้อมพัฒนาโปรแกรม (Python IDE)

สิ่งแวดล้อมสำหรับการพัฒนาแบบเบ็ดเสร็จ (Integrated Development Environment: IDE) คือซอฟต์แวร์ที่ช่วยให้นักพัฒนามีเครื่องมือที่จำเป็นในการเขียน แก้ไข ทดสอบ และดีบั๊กโค้ดในทีเดียวโดย Python IDE ที่คนส่วนใหญ่ใช้กันได้แก่

1) ไพชาร์ม (PyCharm)

JetBrains ซึ่งเป็นบริษัทจากเช็กที่พัฒนาเครื่องมือซอฟต์แวร์ และเป็นผู้สร้างไพชาร์ม (PyCharm) ซึ่งมีรุ่นฟรีสำหรับชุมชนที่เหมาะสมสำหรับแอปพลิเคชันไพทอน (Python) ขนาดเล็ก และรุ่นมืออาชีพแบบมีค่าใช้จ่ายที่เหมาะสมสำหรับการสร้างแอปพลิเคชันไพทอน (Python) ขนาดใหญ่ พร้อมชุดคุณสมบัติทั้งหมดดังต่อไปนี้

- การเติมโค้ดที่เหลือ และการตรวจสอบโค้ดอัตโนมัติ
- การจัดการข้อผิดพลาด และการแก้ไขด่วน
- การทำความสะอาดโค้ดโดยไม่เปลี่ยนฟังก์ชันการทำงาน
- การสนับสนุนเฟรมเวิร์ก (Framework) ของเว็บแอปพลิเคชัน เช่น Django และ Flask
- การสนับสนุนภาษาการเขียนโปรแกรมอื่น ๆ เช่น JavaScript, CoffeeScript, TypeScript, AngularJS และ Node
- เครื่องมือ และไลบรารี (Library) ทางวิทยาศาสตร์ เช่น Matplotlib และ NumPy
- ความสามารถในการเรียกใช้ แก์จุดบกพร่อง ทดสอบ และปรับใช้แอปพลิเคชันในเครื่องเสมือนระยะไกล
- ดีบั๊กเกอร์ (Debugger) เพื่อค้นหาข้อผิดพลาดในโค้ด ตัวสร้างโปรไฟล์เพื่อระบุปัญหาด้านประสิทธิภาพในโค้ด และตัวดำเนินการทดสอบเพื่อเรียกใช้การทดสอบหน่วย
- การรองรับฐานข้อมูลต่าง ๆ

2) ไอดัล (IDLE)

สิ่งแวดล้อมสำหรับการพัฒนา และการเรียนรู้แบบเบ็ดเสร็จ (Integrated Development and Learning Environment: IDLE) คือ Python IDE ที่ติดตั้งตามค่าเริ่มต้น ซึ่งได้รับการพัฒนาด้วยไพทอน (Python) เท่านั้นโดยใช้ชุดเครื่องมือ Tkinter GUI และมีคุณสมบัติดังต่อไปนี้

- ใช้งานได้กับระบบปฏิบัติการมากมาย เช่น Windows, Unix และ macOS
- มีหน้าต่างเชลล์เพื่อเรียกใช้คำสั่ง และแสดงเอาต์พุต (Output)
- นำเสนอตัวแก้ไขข้อความแบบหลายหน้าต่างที่มีการเน้นไวยากรณ์ของโค้ด และการเติมโค้ดที่เหลืออัตโนมัติ
- มีดีบั๊กเกอร์ (Debugger) ของตัวเอง

3) สไปเดอร์ (Spyder)

เป็น IDE แบบโอเพนซอร์ส (Open Source) ที่นักวิทยาศาสตร์ และนักวิเคราะห์ข้อมูลจำนวนมากใช้ ซึ่งมอบประสบการณ์การพัฒนาที่ครอบคลุมพร้อมคุณสมบัติต่าง ๆ สำหรับการวิเคราะห์ข้อมูล การแสดงภาพข้อมูล และการดีบั๊กขั้นสูง นอกจากนี้ยังมีคุณสมบัติดังต่อไปนี้

- ตัวแก้ไขโค้ดที่สมบูรณ์ซึ่งรองรับหลายภาษา
- IPython Console แบบโต้ตอบ
- ดีบั๊กเกอร์ (Debugger) พื้นฐาน
- ไลบรารี (Library) ทางวิทยาศาสตร์ เช่น Matplotlib, SciPy และ NumPy
- ความสามารถในการสำรวจตัวแปรในโค้ด
- ความสามารถในการดูเอกสารประกอบแบบเรียลไทม์ (Realtime)

4) อะตอม (Atom)

เป็นตัวแก้ไขฟรีที่พัฒนาโดย GitHub ซึ่งรองรับการเขียนโค้ดในภาษาการเขียนโปรแกรมหลายภาษา รวมถึงไพทอน (Python) เมื่อใช้ Atom นักพัฒนาจึงสามารถทำงานได้โดยตรงร่วมกับ GitHub ซึ่งเป็นเว็บไซต์ที่คุณสามารถบันทึกโค้ดได้จากส่วนกลาง โดย Atom มีคุณสมบัติดังต่อไปนี้ ได้แก่

- ความสามารถในการใช้ร่วมกับระบบปฏิบัติการหลายระบบ

- การติดตั้งหรือการสร้างแพ็คเกจ (Package) ใหม่ได้ง่าย
- การเติมโค้ดที่เหลื้ออัตโนมัติได้เร็วขึ้น
- ความสามารถในการค้นหาไฟล์ และโครงการ
- การปรับแต่งอินเทอร์เฟซ (Interface) ได้ง่าย

5) ไพทอน เอสดีเค (Python SDK)

ชุดพัฒนาซอฟต์แวร์ (Software Development Kit: SDK) คือชุดเครื่องมือด้านซอฟต์แวร์ที่นักพัฒนาสามารถใช้เพื่อสร้างแอปพลิเคชันซอฟต์แวร์ในภาษาที่กำหนด SDK ส่วนใหญ่มีความเฉพาะเจาะจงสำหรับแพลตฟอร์มฮาร์ดแวร์ และระบบปฏิบัติการที่แตกต่างกัน โดย Python SDK มีเครื่องมือมากมาย เช่น ไลบรารี (Library) ตัวอย่างโค้ด และคู่มือสำหรับนักพัฒนา ซึ่งนักพัฒนาพบว่ามีความประโยชน์เมื่อต้องเขียนแอปพลิเคชัน

2.6 ตัวชี้วัดประสิทธิภาพของแบบจำลอง

การประเมินประสิทธิภาพของแบบจำลองการจำแนกประเภท (Classification Model) นิยมใช้ตารางความสับสน (Confusion Matrix) เพื่อวัดความถูกต้องของการทำนายผลลัพธ์ในแต่ละคลาส โดยแสดงจำนวนตัวอย่างที่แบบจำลองจำแนกได้ถูกต้อง และผิดพลาดในรูปแบบตาราง ซึ่งประกอบด้วยค่าหลัก 4 ค่า ได้แก่ True Positive (TP), True Negative (TN), False Positive (FP) และ False Negative (FN) ซึ่งค่าดังกล่าวสามารถใช้คำนวณตัวชี้วัดทางสถิติที่สำคัญในการประเมินแบบจำลองได้ดังต่อไปนี้

2.6.1 ค่าความถูกต้องโดยรวม (Accuracy)

เป็นตัวชี้วัดที่ใช้วัดสัดส่วนของจำนวนการทำนายที่ถูกต้องทั้งหมดเมื่อเทียบกับจำนวนตัวอย่างทั้งหมดในชุดข้อมูล โดยใช้เพื่อประเมินภาพรวมของประสิทธิภาพของแบบจำลองในการจำแนกผลลัพธ์

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (2.1)$$

2.6.2 ค่าความแม่นยำของการตรวจจับ (Precision)

เป็นตัวชี้วัดที่ใช้ประเมินความถูกต้องเฉพาะกรณีที่แบบจำลองทำนายว่าเป็นคลาสบวก (Positive) โดยพิจารณาว่าผลการทำนายดังกล่าวถูกต้องจริงมากน้อยเพียงใด ใช้สะท้อนความสามารถของแบบจำลองในการลดการทำนายผิดพลาดในกลุ่มที่ไม่ใช่เป้าหมาย

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2.2)$$

2.6.3 ค่าความสามารถในการตรวจจับจริง (Recall)

เป็นตัวชี้วัดที่วัดสัดส่วนของจำนวนข้อมูลที่เป็นคลาสบวก (Positive) จริงทั้งหมดที่แบบจำลองสามารถตรวจจับได้ถูกต้อง ใช้ประเมินความไว (Sensitivity) ของแบบจำลองต่อการตรวจจับคลาสเป้าหมาย

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2.3)$$

2.6.4 ค่าประสิทธิภาพรวมของการตรวจจับ (F1-Score)

เป็นค่าที่แสดงถึงความสมดุลระหว่าง Precision และ Recall โดยคำนวณจากค่าเฉลี่ยเชิงฮาร์โมนิก (Harmonic Mean) ของทั้งสองตัวชี้วัด ซึ่งเหมาะสำหรับใช้ประเมินแบบจำลองที่มีจำนวนข้อมูลในแต่ละคลาสไม่สมดุล

$$\text{F1 - Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.4)$$

ตัวชี้วัดทั้งสี่ค่านี้ช่วยให้สามารถประเมินประสิทธิภาพของแบบจำลองได้ทั้งในเชิงความถูกต้อง ความละเอียด และความสมดุลของผลลัพธ์ ซึ่งเป็นปัจจัยสำคัญในการเลือก และปรับปรุงแบบจำลองให้มีความเหมาะสมต่อการใช้งานในสถานการณ์จริง

2.7 งานวิจัยที่เกี่ยวข้อง

2.7.1 แนวทางการใช้วิสัยทัศน์ของเครื่อง YOLOv3 ในการตรวจสอบการปิดตาสำหรับการตรวจจับการขับรถเมื่อง่วง [11]

งานวิจัยนี้มีเป้าหมายเพื่อพัฒนาระบบตรวจจับอาการง่วงของผู้ขับขี่รถยนต์แบบเรียลไทม์เพื่อลดอุบัติเหตุบนท้องถนนซึ่งเป็นปัญหาที่รุนแรงทั่วโลกโดยมีผู้เสียชีวิตมากกว่า 1.25 ล้านคนต่อปี หนึ่งในสาเหตุหลักของอุบัติเหตุคือการขับรถในสภาวะง่วงหรืออ่อนเพลียซึ่งส่งผลต่อการตัดสินใจ และการตอบสนองของผู้ขับ งานนี้จึงมุ่งเน้นการใช้เทคโนโลยีปัญญาประดิษฐ์หรือ DL เพื่อวิเคราะห์ภาพใบหน้า และดวงตาของผู้ขับขี่เพื่อตรวจสอบสภาวะของดวงตาว่าอยู่ในสภาพปิดหรือตื่น ระบบนี้ออกแบบให้สามารถตรวจจับได้แบบเรียลไทม์ (Realtime) โดยใช้การประมวลผลภาพ (Image Processing) ที่แม่นยำ และรวดเร็วเพื่อลดความเสี่ยงของอุบัติเหตุจากการหลับในของผู้ขับ

1) แนวคิดของระบบตรวจจับความง่วง

ระบบตรวจจับอาการง่วงสามารถแบ่งได้เป็นสองประเภทคือระบบที่มุ่งเน้นยานพาหนะ (Vehicle-Oriented System) และ ระบบที่มุ่งเน้นผู้ขับขี่ (Driver-Oriented System) ประเภทแรกใช้ข้อมูลจากเซ็นเซอร์ของรถยนต์เช่นการเลี้ยว ความเร็ว การเหยียบเบรกหรือคันเร่ง เพื่อวิเคราะห์พฤติกรรมของผู้ขับซึ่งมีข้อจำกัดคือพฤติกรรมของแต่ละคนแตกต่างกันมากทำให้โมเดลไม่สามารถใช้ได้กับทุกคน ส่วนประเภทที่สองคือ ระบบที่มุ่งเน้นผู้ขับขี่ (Driver-Oriented System) ซึ่งเน้นการตรวจจับลักษณะทางกายภาพของผู้ขับเช่นการเคลื่อนไหวของใบหน้า ดวงตา หรือปาก งานวิจัยนี้เลือกใช้วิธีหลังเพราะสามารถตรวจสอบอาการง่วงได้โดยตรงผ่านการวิเคราะห์การปิดตาซึ่งเป็นตัวบ่งชี้ที่แม่นยำที่สุด และเหมาะกับการใช้งานในสภาพแวดล้อมจริงมากกว่า

2) โครงสร้างการทำงานของระบบ (System Framework)

ระบบทำงานเป็น 4 ขั้นตอนคือ การเตรียมข้อมูล การฝึกโมเดล การประเมินผล และการทดสอบจริง ใช้ YOLOv3 เป็นอัลกอริทึมหลักในการตรวจจับวัตถุ (ดวงตา) ภาพที่ได้จะผ่านการประมวลผลแบบเรียลไทม์เพื่อแยกว่าผู้ขับตาปิดหรือตาเปิด

3) การเตรียมข้อมูล (Dataset Preparation)

การรวบรวมชุดข้อมูลเป็นขั้นตอนแรกของการสร้างระบบตรวจจับ โดยผู้วิจัยได้ใช้ภาพจากแหล่งข้อมูลออนไลน์ทั้งกูเกิล อิมเมจส์ (Google Images) และเว็บไซต์เค็กเกิล (Kaggle) รวมทั้งหมดจำนวน 1,000 ภาพ ซึ่งแบ่งออกเป็น 80% สำหรับการฝึกสอน (Training) และอีก 20%

สำหรับการตรวจสอบความถูกต้อง (Validation) นอกจากนี้ยังเพิ่มภาพอีก 10 ภาพ และวิดีโอหนึ่งชุดเพื่อใช้ในการทดสอบจริงเพื่อให้เห็นประสิทธิภาพของโมเดลกับข้อมูลที่ไม่เคยเห็นมาก่อน ดังรูปที่ 2.6 ภาพทั้งหมดจะถูกนำมาใช้ในการฝึกโมเดล YOLOv3 เพื่อให้สามารถระบุได้ว่าดวงตาในภาพอยู่ในสภาวะเปิดหรือว่าง



รูปที่ 2.6 การเตรียมข้อมูล (Dataset Preparation) [11]

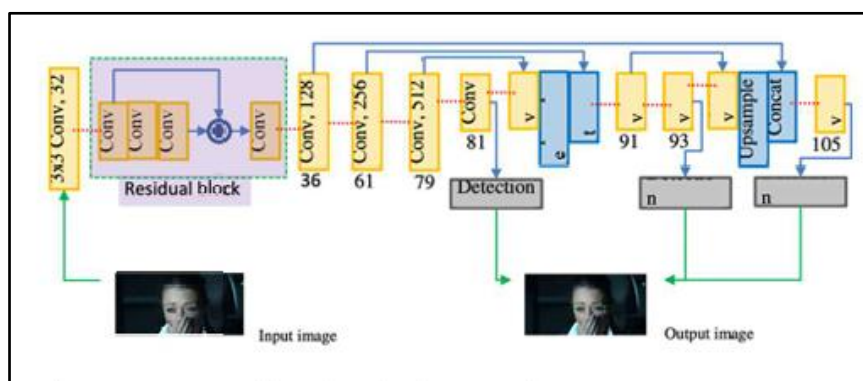
4) การติดป้ายข้อมูล (Data Annotation)

ใช้โปรแกรม Labellmg ในการสร้างกรอบรอบดวงตา (Bounding Box) และบันทึกพิกัดในรูปแบบ Pascal VOC เพื่อใช้ในการฝึก YOLOv3 ให้รู้ว่ามีบริเวณใดคือดวงตาที่ต้องตรวจจับ YOLOv3 ใช้โครงสร้าง Darknet-53 มี 53 ชั้น Convolutional Layer ทำงานเร็ว และแม่นยำสูง ใช้เทคนิค Feature Pyramid ตรวจจับวัตถุหลายขนาดในภาพได้ดี เหมาะกับงานตรวจจับดวงตาซึ่งมีขนาดเล็ก และเปลี่ยนแปลงตามมุมกล้อง

5) การฝึก และประเมินผลโมเดล (Training and Evaluation)

ฝึกใน Google Colab ทั้งหมด 50 Epoch โดยวัดค่าความแม่นยำจากค่าเฉลี่ยความแม่นยำเฉลี่ย (Mean Average Precision: mAP) ผลการฝึกได้ mAP สูงสุด 100% ใน Model 20 มีค่า Train Loss 2.59 และ Val Loss 1.40 แสดงว่าโมเดลเรียนรู้ได้ดี และไม่โอเวอร์ฟิตติ้ง (Overfitting) เมื่อนำไปทดสอบกับภาพ และวิดีโอใหม่ ระบบสามารถตรวจจับการปิดตาได้ถูกต้อง

ทั้งหมด ความแม่นยำต่อเฟรมอยู่ระหว่าง 40–89% แต่ภาพรวมทดสอบได้ถูกต้อง 100% ยืนยันว่าระบบสามารถใช้งานจริงได้ในสถานการณ์ขับขี่ ดังรูปที่ 2.7



รูปที่ 2.7 โครงสร้างตรวจจับวัตถุ [11]

6) สรุปผลการศึกษา (Conclusion)

ผลลัพธ์ของการศึกษาแสดงให้เห็นว่าเทคนิค DL โดยเฉพาะ YOLOv3 สามารถนำมาใช้ในการตรวจจับอาการง่วงของผู้ขับขี่ได้อย่างมีประสิทธิภาพสูง โมเดลที่สร้างขึ้นมีค่าความแม่นยำในการฝึก 97.36% ความแม่นยำในการตรวจสอบ 98.60% และความแม่นยำในการทดสอบจริงถึง 100% การใช้โมเดลแบบ Pre-trained ทำให้ไม่ต้องออกแบบโครงสร้างใหม่ทั้งหมดและสามารถทำงานได้แบบเรียลไทม์ซึ่งเหมาะสมกับระบบตรวจจับความง่วงในรถยนต์ ระบบนี้สามารถต่อยอดได้อีกมากทั้งในด้านการเพิ่มข้อมูล และการเชื่อมต่อกับอุปกรณ์จริงเพื่อช่วยลดอุบัติเหตุจากการหลับในของผู้ขับ

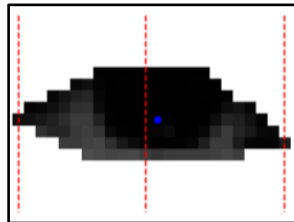
2.7.2 ระบบตรวจสอบการขับขี่เกี่ยวกับสมาธิโดยใช้อัลกอริทึมการตัดสินใจในการขับขี่ [12]

งานวิจัยนี้มุ่งเน้นพัฒนาระบบตรวจจับความประมาท และความง่วงของผู้ขับขี่ (Driver Monitoring System: DMS) ซึ่งถือเป็นสาเหตุหลักของอุบัติเหตุบนท้องถนน โดยเฉพาะในเกาหลีใต้ที่จากสถิติปี 2015–2019 พบว่า 67.6% ของอุบัติเหตุเกิดจากการง่วง และการวอกแวกขณะขับรถ งานวิจัยจึงเสนออัลกอริทึมใหม่ชื่อว่า อัลกอริทึมการตัดสินใจขณะขับขี่ที่ถูกรบกวน (Distracted Driving Decision Algorithm: DDDA) เพื่อใช้ตรวจจับพฤติกรรมดังกล่าวแบบเรียลไทม์ จุดเด่นของระบบนี้คือไม่ใช่โมเดล ML แต่ใช้วิธีคำนวณเชิงคณิตศาสตร์ร่วมกับเทคนิคหน้าต่างเลื่อน (Sliding Window Technique) ทำให้มีประสิทธิภาพสูง และใช้ทรัพยากรน้อย

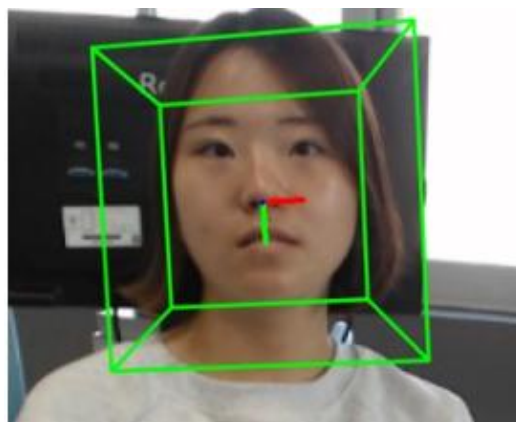
เหมาะกับการใช้งานในรถยนต์ที่มีข้อจำกัดด้านฮาร์ดแวร์ ผลลัพธ์จากการทดลองแสดงว่า อัลกอริทึมนี้ทำงานได้แม่นยำเฉลี่ย 83.5% และประมวลผลต่อเฟรมได้ในเวลาเพียง 42 มิลลิวินาที

1) วิธีการทำงานของระบบ (Proposed Method)

ระบบ DDDA ทำงานแบบเรียลไทม์โดยประมวลผลวิดีโอจากกล้องติดหน้ารถเพื่อเฝ้าระวังพฤติกรรมที่ไม่ปลอดภัย เช่น การหลับตานานกว่า 3 วินาที การก้มหน้า หรือหันมองข้างทาง เมื่อกล้องตรวจพบใบหน้า ระบบจะระบุตำแหน่งดวงตา จมูก และปากเพื่อใช้ในการประเมินการกะพริบตา ทิศทางการมอง และการเอียงศีรษะ ขั้นตอนเริ่มจากการตรวจจับใบหน้าและจุด Landmark ด้วย DLIB และติดตามต่อเนื่องระหว่างเฟรม จากนั้นจึงทำการตรวจจับการกะพริบตา (Blink Detection) โดยคำนวณอัตราส่วนความกว้าง และความสูงของดวงตาเพื่อระบุการหลับตา ดังแสดงในรูปที่ 2.8 ถัดมาคือ Head Pose Estimation ซึ่งใช้จุด Landmark ประเมินแกนสามมิติ Yaw, Pitch และ Roll เพื่อบอกทิศทางศีรษะ และสุดท้ายระบบจะตรวจจับการก้มศีรษะจากอัตราส่วนระยะจมูก และคาง ดังแสดงในรูปที่ 2.9



รูปที่ 2.8 ผลการประเมินตำแหน่ง และการจ้องมองของรูม่านตา [12]



รูปที่ 2.9 ผลการประเมินท่าทางศีรษะ [12]

2) การประมวลผลผลลัพธ์ (Post Processing)

ระบบ DDDA จะรวมผลลัพธ์แบบไบนารี (Binary) จากทั้งสองส่วนคือ การหลับตา การมองออกด้านข้าง การหันศีรษะ และการก้มหน้า เพื่อสรุปสถานะของผู้ขับ การคำนวณจะใช้เทคนิคหน้าต่างเลื่อน (Sliding Window Technique) ซึ่งเป็นการเก็บค่าผลลัพธ์จากหลายเฟรมล่าสุดมาวิเคราะห์ร่วมกันเพื่อหลีกเลี่ยงการแจ้งเตือนผิดพลาดจากเฟรมเดียว เช่น หากดวงตาปิดติดต่อกันใน 80% ของ 90 เฟรมล่าสุด ระบบจะตัดสินใจผู้ขับง่วงหรือไม่ได้มองทาง วิธีนี้ช่วยลดการใช้หน่วยความจำ และเพิ่มความเสถียรของผลลัพธ์

3) สรุป และข้อเสนอแนะ (Conclusion & Future Work)

อัลกอริทึม DDDA สามารถตรวจจับพฤติกรรมของผู้ขับได้ทั้งการหลับตา การก้มหน้า การหันศีรษะ และการมองข้างในช่วงเวลามากกว่า 3 วินาที พร้อมส่งสัญญาณเตือนแบบเรียลไทม์ จุดเด่นคือไม่ใช้โมเดลเรียนรู้เชิงลึกแต่ใช้การคำนวณที่มีประสิทธิภาพจึงใช้ทรัพยากรน้อย และเหมาะกับระบบฝังตัวในรถยนต์ อย่างไรก็ตามยังมีข้อจำกัดเมื่อใบหน้าถูกบังด้วยมือหรือโทรศัพท์ หรือกรณีที่ศีรษะก้มต่ำเกินไปทำให้ตรวจจับจุด Landmark ไม่ครบ ทีมวิจัยเสนอให้ปรับปรุงการตรวจจับใบหน้าให้แม่นยำขึ้นเพื่อเพิ่มประสิทธิภาพโดยรวมของระบบในอนาคต

2.7.3 ระบบตรวจสอบสถานการณ์ง่วงนอนโดยอาศัยการวิเคราะห์ลักษณะใบหน้า [13]

แนวคิดของงานวิจัยงานนี้มีเป้าหมายในการสร้างระบบเฝ้าระวังอาการง่วงของผู้ขับที่ใช้โดยอาศัยการวิเคราะห์ลักษณะใบหน้า ดวงตา และการเคลื่อนไหวศีรษะแบบเรียลไทม์ ระบบไม่ได้ใช้โมเดล DL ขนาดใหญ่แต่ใช้การคำนวณเชิงภาพ ทำให้สามารถทำงานได้เร็ว และใช้ทรัพยากรน้อย เหมาะสำหรับติดตั้งในรถยนต์จริง

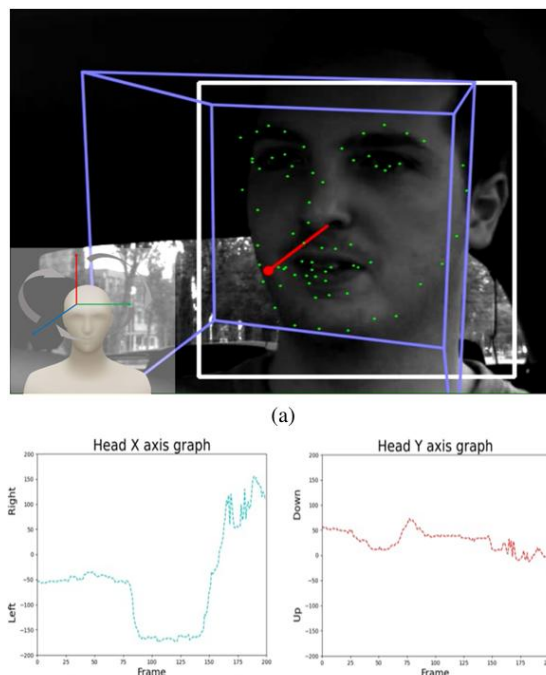
1) วิธีการทำงานของระบบ และขั้นตอนการประมวลผล

ระบบเริ่มจากตรวจจับใบหน้าผู้ขับด้วยโมเดล RetinaFace เพื่อหาตำแหน่ง และจุด Landmark ของดวงตา และส่วนต่าง ๆ บนใบหน้า จากนั้นนำจุด Landmark ของตาไปวิเคราะห์โดยใช้เทคนิค Adaptive Threshold เพื่อแยกแยะตาออกจากภาพ ซึ่งช่วยให้ตรวจจับได้แม่นยำแม้ในสภาพแสงที่เปลี่ยนหรือศีรษะเอียง นอกจากนี้ยังใช้ PnP Algorithm เพื่อคำนวณทิศทางของศีรษะว่าก้ม เงย หรือหันไปด้านใด ระบบจะตรวจสอบพฤติกรรมของดวงตา และศีรษะต่อเนื่องในแต่ละเฟรม หากพบว่าดวงตาปิดนานกว่าค่าที่กำหนด หรือศีรษะพยักหลังบ่อยครั้ง จะ

ประเมินว่าผู้ขับเริ่มมีอาการง่วงหรือสูญเสียสมาธิ ระบบนี้สามารถระบุระดับความง่วงได้ 3 ระดับ ได้แก่ ปกติ ง่วงเล็กน้อย และง่วงมาก

2) ผลการทดสอบ และประสิทธิภาพ

ผลการทดลองแสดงให้เห็นว่าระบบสามารถทำงานแบบเรียลไทม์ และตรวจจับได้แม่นยำสูง ทั้งในสภาพแสงแตกต่างกัน และการเคลื่อนไหวศีรษะเล็กน้อย ระบบมีการประมวลผลรวดเร็ว และใช้หน่วยความจำต่ำ เหมาะกับการทำงานบนอุปกรณ์ฝังตัวงานนี้ช่วยเพิ่มความปลอดภัยในการขับขี่โดยไม่ต้องใช้โมเดล AI ขนาดใหญ่ สามารถนำไปต่อยอดในระบบกล้องภายในรถยนต์หรือระบบ IoT ที่เฝ้าระวังผู้ขับขี่ได้จริง ช่วยลดอุบัติเหตุจากการหลับใน และอาการเหนื่อยล้า ดังรูปที่ 2.10



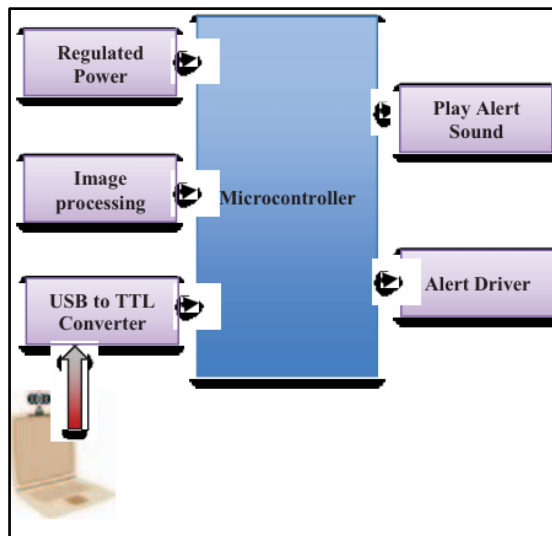
รูปที่ 2.10 ผลการทดลองการประมาณทิศทางโดยอาศัยท่าทางศีรษะ [13]

2.7.4 ระบบเตือนผู้ขับซึ่งง่วงนอนโดยใช้ OpenCV [14]

แนวคิดของงานวิจัยงานนี้พัฒนาโปรแกรมตรวจจับ และเตือนอาการง่วงของผู้ขับโดยใช้คอมพิวเตอร์วิทัศน์ (Computer Vision) ผ่าน OpenCV จุดเด่นคือระบบเรียบง่าย ไม่ต้องการฝึกโมเดล AI แต่สามารถตรวจจับอาการหลับตา และการหาได้อย่างมีประสิทธิภาพ

1) ขั้นตอนการทำงาน

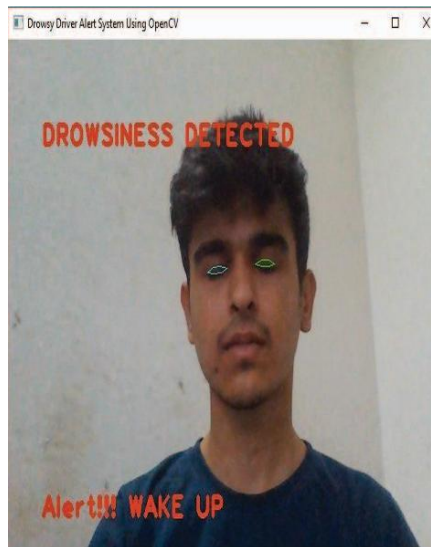
ระบบเริ่มจากกล้องจับภาพใบหน้าของผู้ขับขี่ จากนั้นตรวจหาตำแหน่งจุด Landmark ของดวงตา และปากด้วย DLIB Library แล้วคำนวณค่า EAR เพื่อประเมินการเปิดหรือปิดของดวงตา หากค่า EAR ต่ำกว่าค่ากำหนด และคงอยู่เกินระยะเวลาที่ตั้งไว้ ระบบจะตีความว่า ผู้ขับหลับตา ดังรูปที่ 2.11



รูปที่ 2.11 แผนภาพการทำงานของระบบ [14]

2) การตรวจจับการหาว และการเตือน

นอกจากการหลับตา ระบบยังวิเคราะห์ขนาดการเปิดของปากจากจุด Landmark รอบริมฝีปากเพื่อระบุการหาว ซึ่งเป็นสัญญาณของความง่วง เมื่อพบอาการดังกล่าว ระบบจะเปิดเสียงเตือนทันทีเพื่อให้ผู้ขับรู้ตัว และหยุดพัก จากการทดสอบในสภาพแสงทั่วไป และการเคลื่อนไหวศีรษะปกติ ระบบสามารถตรวจจับการหลับตา และการหาวได้อย่างแม่นยำสูง ตอบสนองได้รวดเร็ว การประมวลผลไม่หน่วง และทำงานได้ต่อเนื่องแบบเรียลไทม์ ดังรูปที่ 2.12



รูปที่ 2.12 Drowsiness Detection [14]

3) จุดเด่น และการใช้งานจริง

ระบบนี้ไม่ซับซ้อน ใช้เพียงกล้อง USB และไมโครคอนโทรลเลอร์พื้นฐาน สามารถต่อยอดเป็นอุปกรณ์เตือนผู้ขับในรถยนต์ทั่วไปได้ทันที และเหมาะกับโครงการต้นแบบด้านความปลอดภัย

2.7.5 การตรวจจับอาการง่วงนอนขณะขับรถโดยการหลับตา และการหา ระบบวิสัยทัศน์ที่กำหนดเองของ Azure [15]

แนวคิดของงานวิจัยงานนี้ใช้เทคโนโลยี DL บนแพลตฟอร์ม Microsoft Azure Custom Vision เพื่อสร้างระบบตรวจจับความง่วงของผู้ขับจากการหลับตา และหา จุดเด่นคือเป็นระบบที่ใช้งานง่าย ไม่ต้องเขียนโค้ดมากเพราะใช้ระบบ Cloud Auto-Training ของ Azure

1) การเตรียมข้อมูล และโมเดลที่ใช้

ใช้ชุดข้อมูลภาพจำนวนกว่า 2,900 ภาพจาก Kaggle ซึ่งแบ่งเป็นภาพผู้ขับที่ขณะตื่น และ ง่วง เพื่อนำมาเทรนโมเดล VGG16 เครือข่ายประสาทเทียมแบบคอนโวลูชัน (Convolutional Neural Network: CNN) ที่มีความสามารถสูงในการจำแนกภาพ ระบบนี้สามารถฝึก และปรับแต่งโมเดลได้บน Cloud โดยไม่ต้องใช้เครื่องที่มี GPU ดังรูปที่ 2.13



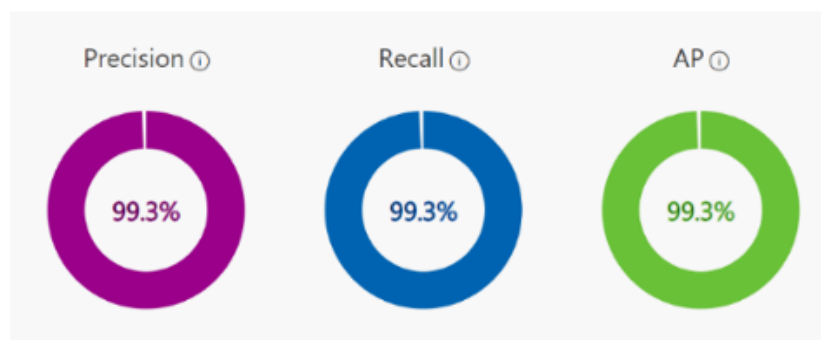
รูปที่ 2.13 ชุดข้อมูลตัวอย่างสำหรับการตรวจจับอาการง่วงนอนของผู้ขับขี่ [15]

2) การประมวลผล และการทดสอบระบบ

หลังการฝึก โมเดลจะถูกนำมาทดสอบกับภาพใหม่ ระบบสามารถจำแนกได้ว่าเป็น Awake หรือ Drowsy พร้อมบอกค่าเปอร์เซ็นต์ความมั่นใจของผลลัพธ์ และยังสามารถตรวจจับทั้งการหลับตา และการหาวในภาพนิ่งแบบเรียลไทม์

3) ผลลัพธ์ และความแม่นยำ

ผลการทดสอบแสดงว่าโมเดลมีความแม่นยำสูงถึง 99.3% ซึ่งเหนือกว่าโมเดลทั่วไปอย่าง CNN หรือ เครือข่ายประสาทเทียมแบบวนซ้ำ (Recurrent Neural Network: RNN) ทั้งในด้านความเร็ว และความถูกต้อง ดังรูปที่ 2.14 ถือเป็นระบบที่ให้ผลลัพธ์ดีที่สุดในงานประเภทเดียวกัน เหมาะสำหรับการนำไปพัฒนาเป็นแอปพลิเคชันตรวจจับผู้ขับในรถยนต์หรืออุปกรณ์ IoT เช่น กล้องอัจฉริยะในรถ สามารถปรับใช้กับสภาพแวดล้อมจริงได้ง่าย และต่อยอดสู่ระบบตรวจจับอัตโนมัติในรถยนต์อัจฉริยะได้



รูปที่ 2.14 ประสิทธิภาพของแบบจำลองที่เสนอ [15]

บทที่ 3

วิธีการดำเนินงาน

การศึกษานี้ได้ดำเนินการออกแบบ และพัฒนาระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ โดยมุ่งเน้นการตรวจจับพฤติกรรมที่บ่งชี้ถึงภาวะง่วงนอนจากภาพวิดีโอแบบเรียลไทม์ ขั้นตอนการดำเนินงานประกอบด้วย การออกแบบระบบโดยรวม การออกแบบอัลกอริทึม การฝึกสอนอัลกอริทึมด้วยชุดข้อมูล และการทดสอบการทำงานของระบบแจ้งเตือน เพื่อประเมินประสิทธิภาพ และความถูกต้องของการทำงานของระบบ

3.1 แผนการดำเนินงาน

แผนการดำเนินงานของโครงการนี้ได้ถูกวางลำดับขั้นตอนอย่างเป็นระบบ เพื่อให้การดำเนินงานเป็นไปอย่างมีประสิทธิภาพ และสามารถบรรลุวัตถุประสงค์ที่ตั้งไว้ได้ โดยเริ่มจากการศึกษาทฤษฎี และแนวคิดที่เกี่ยวข้องกับหัวข้อโครงการ เพื่อสร้างพื้นฐานความเข้าใจในหลักการทำงานของระบบ และเทคโนโลยีที่เกี่ยวข้อง จากนั้นจึงศึกษารูปแบบการเขียนโปรแกรม และอัลกอริทึมที่เกี่ยวข้องกับปัญญาประดิษฐ์ ซึ่งเป็นหัวใจสำคัญของโครงการ เพื่อให้สามารถนำมาประยุกต์ใช้ในการออกแบบ และพัฒนาโปรแกรมได้อย่างเหมาะสม

เมื่อมีความเข้าใจพื้นฐานแล้ว จึงเข้าสู่ขั้นตอนการวิเคราะห์ และออกแบบระบบ โดยกำหนดโครงสร้างการทำงานของระบบในเชิงลำดับขั้น การกำหนดอินพุต เอาต์พุต และการประมวลผลภายในระบบให้สอดคล้องกับวัตถุประสงค์ของการพัฒนา ต่อมาจะเป็นการเก็บรวบรวมข้อมูลที่จำเป็น และดำเนินการเขียนอัลกอริทึมตามแบบจำลองที่ออกแบบไว้ ซึ่งในขั้นตอนนี้จะต้องมีการตรวจสอบความถูกต้องของข้อมูล และทดสอบอัลกอริทึมในเบื้องต้นเพื่อให้แน่ใจว่าสามารถทำงานได้ตามที่คาดหวัง

หลังจากนั้นจะเข้าสู่กระบวนการทดสอบระบบจริง และบันทึกผลการทดลองอย่างเป็นระบบ เพื่อใช้ในการวิเคราะห์ประสิทธิภาพของโปรแกรม รวมถึงการปรับปรุงแก้ไขข้อบกพร่องที่พบระหว่างการทดสอบ เมื่อระบบสามารถทำงานได้อย่างถูกต้อง และเสถียรแล้ว จึงดำเนินการสรุปผลการดำเนินงานทั้งหมด ทั้งในด้านเทคนิค และผลลัพธ์เชิงประสิทธิภาพ ก่อนจัดทำเอกสารรายงานฉบับสมบูรณ์ในรูปแบบปริญญานิพนธ์ และนำเสนอผลการดำเนินงานต่อคณะกรรมการเพื่อตรวจสอบ และประเมินผลการดำเนินโครงการ ซึ่งแสดงไว้ดังรูปที่ 3.1 และตารางที่ 3.1 ดังนี้



รูปที่ 3.1 บล็อกไดอะแกรมการดำเนินงาน

ตารางที่ 3.1 แผนการดำเนินโครงการ

ขั้นตอนการดำเนินงาน	พ.ศ. 2569		พ.ศ. 2569									
	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย.	พ.ค.	มิ.ย.	ก.ค.	ส.ค.	ก.ย.	ต.ค.
เสนอหัวข้อโครงการ												
ศึกษาทฤษฎีที่เกี่ยวข้อง												
ศึกษาการเขียนโปรแกรมอัลกอริทึมของปัญญาประดิษฐ์												
วิเคราะห์และออกแบบชิ้นงาน												
การเก็บตัวอย่างจากผลการทดสอบและเขียนอัลกอริทึม												
ทดสอบและบันทึกผล												
สรุปผลการดำเนินงาน												
ทำปริญญานิพนธ์												
นำเสนอผลงาน												

----- แผนการดำเนิน

_____ การดำเนินการจริง

3.2 การออกแบบระบบ

ส่วนของการออกแบบของระบบการประยุกต์ปัญญาประดิษฐ์สำหรับประเมินผลระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ มีดังนี้

3.2.1 แนวคิดการทำงานระบบ

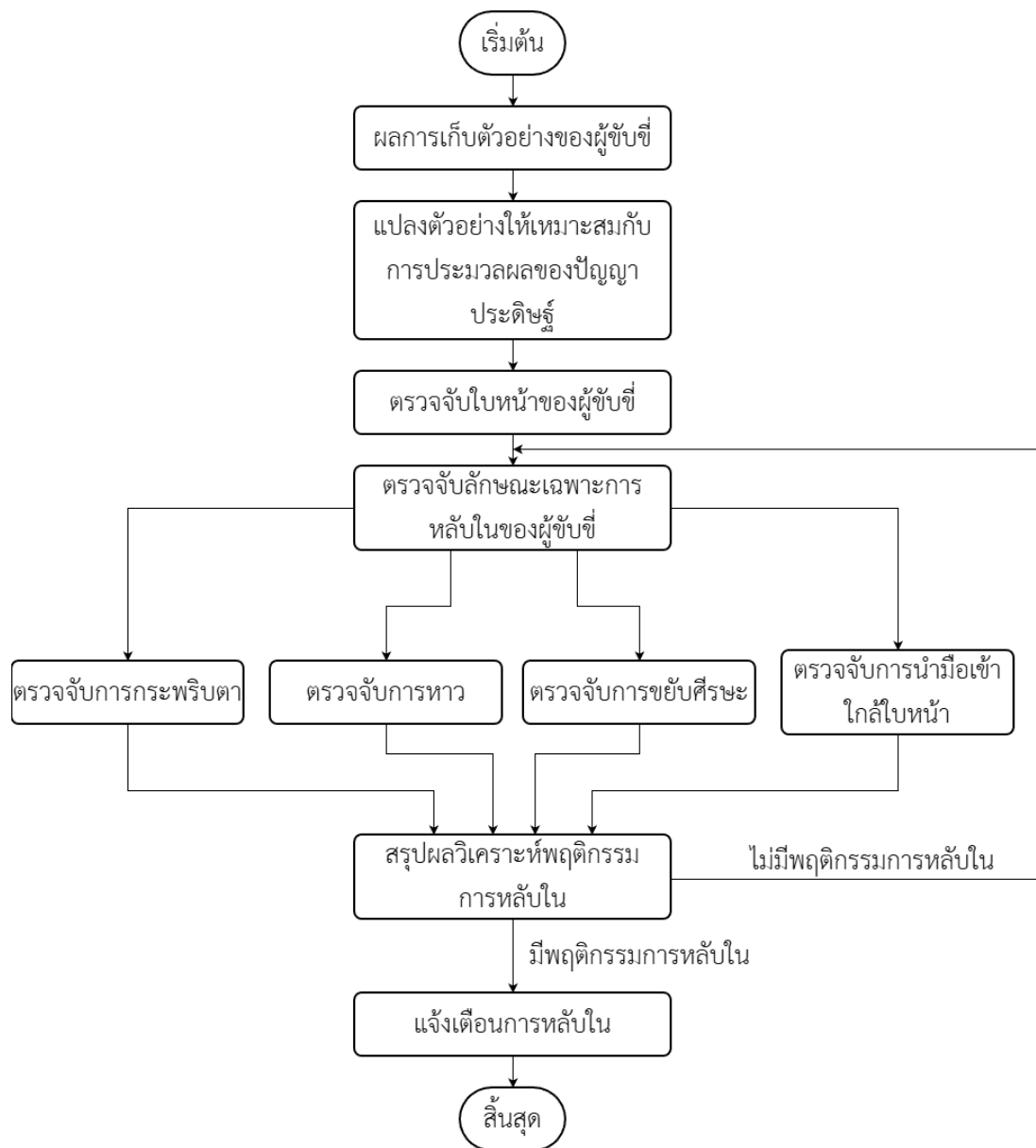
เพื่อประยุกต์ใช้ปัญญาประดิษฐ์สำหรับประเมินภาวะความง่วงของผู้ขับขี่ยานพาหนะ โดยใช้เทคนิคการประมวลผลภาพ และการเรียนรู้ของเครื่องในการตรวจจับพฤติกรรมของผู้ขับ เช่น การกะพริบตา การหาว และการพงกศีรษะเพื่อวิเคราะห์ และจำแนกสถานะของผู้ขับว่าอยู่ในภาวะตื่นหรือหลับใน

ระบบใช้โมเดลปัญญาประดิษฐ์แบบ LSTM ในการวิเคราะห์ลำดับภาพต่อเนื่องแบบเรียลไทม์ และมีระบบเสียงแจ้งเตือนอัตโนมัติเมื่อพบภาวะง่วงเพื่อช่วยลดความเสี่ยงจากอุบัติเหตุ และสามารถนำไปประยุกต์ใช้กับยานพาหนะได้จริงในอนาคต

3.2.2 การออกแบบระบบการทำงาน

การออกแบบระบบการทำงานของระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะด้วยปัญญาประดิษฐ์ ได้กำหนดกระบวนการทำงานอย่างเป็นขั้นตอนเพื่อให้ระบบสามารถตรวจจับ และประมวลผลพฤติกรรมของผู้ขับได้อย่างถูกต้อง โดยเริ่มจากการเก็บตัวอย่างภาพของผู้ขับจากกล้องเว็บแคม จากนั้นจึงทำการแปลง และประมวลผลภาพให้อยู่ในรูปแบบที่เหมาะสมต่อการวิเคราะห์ด้วยอัลกอริทึมของปัญญาประดิษฐ์ ระบบจะตรวจจับใบหน้าของผู้ขับที่เป็นลำดับแรก เพื่อระบุตำแหน่งของอวัยวะสำคัญบนใบหน้า เช่น ดวงตา ปาก และศีรษะ แล้วดำเนินการตรวจจับลักษณะเฉพาะที่บ่งบอกถึงอาการหลับใน เช่น การกะพริบตา การหาว การก้มศีรษะ หรือการนำมือเข้าใกล้ใบหน้า ซึ่งเป็นพฤติกรรมที่สัมพันธ์กับอาการง่วงนอน

เมื่อระบบตรวจจับข้อมูลจากแต่ละส่วนแล้ว จะทำการวิเคราะห์ และสรุปผลรวมของพฤติกรรม เพื่อประเมินว่าผู้ขับที่มีภาวะหลับในหรือไม่ หากระบบไม่พบพฤติกรรมดังกล่าว จะดำเนินการตรวจจับซ้ำอย่างต่อเนื่อง แต่หากพบว่ามีพฤติกรรมที่เข้าข่ายหลับใน ระบบจะส่งสัญญาณแจ้งเตือนทันทีเพื่อกระตุ้นให้ผู้ขับที่กลับมามีสมาธิในการขับรถ กระบวนการทั้งหมดนี้ทำงานแบบเรียลไทม์โดยใช้กล้องเว็บแคมร่วมกับโมเดลปัญญาประดิษฐ์ที่ได้รับการฝึกฝนล่วงหน้า เพื่อให้สามารถทำงานได้อย่างรวดเร็ว และแม่นยำ ดังรูปที่ 3.2



รูปที่ 3.2 โฟลวชาร์ตลำดับการประเมินผลของปัญญาประดิษฐ์

3.3 การเก็บตัวอย่างผลการทดสอบ

การเก็บตัวอย่างผลการทดสอบเป็นขั้นตอนสำคัญที่มีผลโดยตรงต่อความถูกต้อง และความน่าเชื่อถือของโมเดลปัญญาประดิษฐ์ที่พัฒนาขึ้น ในโครงการนี้ได้กำหนดให้เก็บข้อมูลจากการทดสอบด้วยกล้องบันทึกวิดีโอ เพื่อใช้สำหรับตรวจจับอาการหลับในของผู้ขับขี่ โดยข้อมูลที่ได้อาจถูกนำมาผ่านกระบวนการประมวลผลเบื้องต้น เพื่อให้ได้ชุดข้อมูลที่พร้อมใช้งานสำหรับการฝึกสอน (Training) การตรวจสอบความถูกต้อง (Validation) และการทดสอบ (Testing) ของอัลกอริทึมปัญญาประดิษฐ์

3.3.1 การเก็บข้อมูลจากการถ่ายทำด้วยตนเอง

ผู้จัดทำได้ดำเนินการบันทึกวิดีโอจากสถานการณ์จำลองการขับขีภายในรถยนต์ โดยใช้กล้องจากโทรศัพท์ที่มีความละเอียดไม่ต่ำกว่า 720p เพื่อให้ได้ภาพใบหน้า และท่าทางของผู้ขับขีที่ชัดเจน การบันทึกดำเนินการในหลายสภาวะของผู้ขับขี ได้แก่

- 1) สภาวะตื่นตัว (Awake) ผู้ขับขีลืมตา และมีสมาธิในการขับรถ
- 2) สภาวะง่วง (Drowsy) ผู้ขับขีมีการกระพริบตาถี่ หาว หรือพยักหน้าช้า ๆ

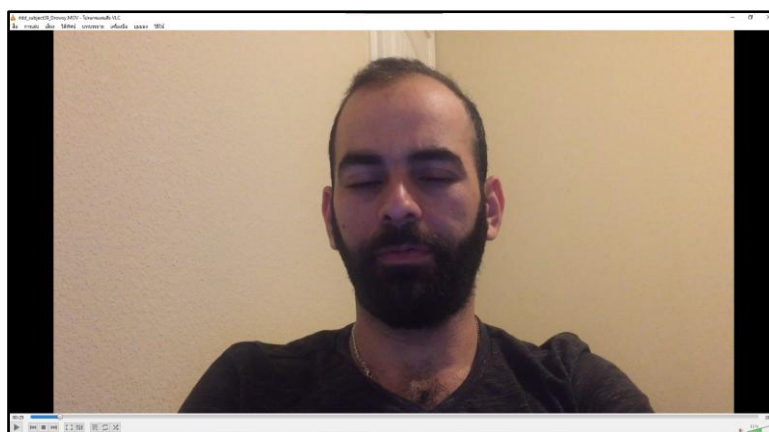
หลังการบันทึกวิดีโอจะมีการคัดกรองเฉพาะเฟรมที่เห็นใบหน้าอย่างชัดเจน และจัดเก็บในรูปแบบไฟล์ .mp4 พร้อมระบุรหัสวิดีโอ และสถานะของผู้ขับขีแต่ละรายอย่างชัดเจน เพื่อป้องกันการซ้ำซ้อนของข้อมูลในแต่ละชุด ดังรูปที่ 3.3

รูปที่ 3.3 ตัวอย่างการถ่ายทำด้วยตนเอง

3.3.2 การเก็บข้อมูลจากแหล่งข้อมูลสาธารณะ

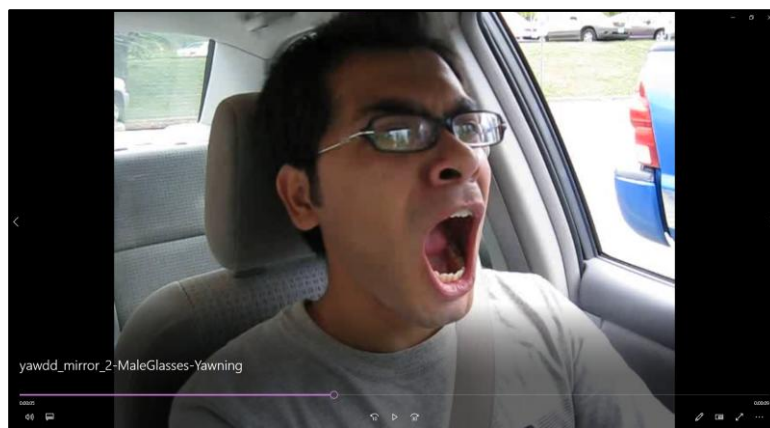
นอกจากการเก็บข้อมูลจากการถ่ายทำด้วยตนเองแล้ว โครงการนี้ยังได้รวบรวมข้อมูลจากฐานข้อมูลสาธารณะที่เปิดให้ใช้งานเพื่อการวิจัย เพื่อเพิ่มความหลากหลายของบุคคล มุมกล้อง และสภาพแสง ซึ่งเป็นปัจจัยสำคัญที่ช่วยให้โมเดลปัญญาประดิษฐ์สามารถเรียนรู้ได้อย่างครอบคลุม และลดความเอนเอียงของข้อมูล เลือกใช้ชุดข้อมูลที่มีความน่าเชื่อถือ และได้รับการอ้างอิงในงานวิจัยก่อนหน้า จำนวน 3 แหล่ง ดังนี้

- 1) UTAR Real-Life Drowsiness Dataset (UTA-RLDD) ชุดข้อมูลนี้พัฒนาโดย University of Texas at Arlington (UTA) ประเทศมาเลเซีย ประกอบด้วยวิดีโอของผู้ขับขีในสภาวะจริงขณะเกิดอาการง่วงนอน โดยมีการระบุระดับความง่วงอย่างชัดเจน (Awake, Drowsy และ Very Drowsy) ซึ่งเหมาะสำหรับใช้ในการฝึกสอนโมเดลตรวจจับอาการง่วงจากใบหน้า ดังรูปที่ 3.4



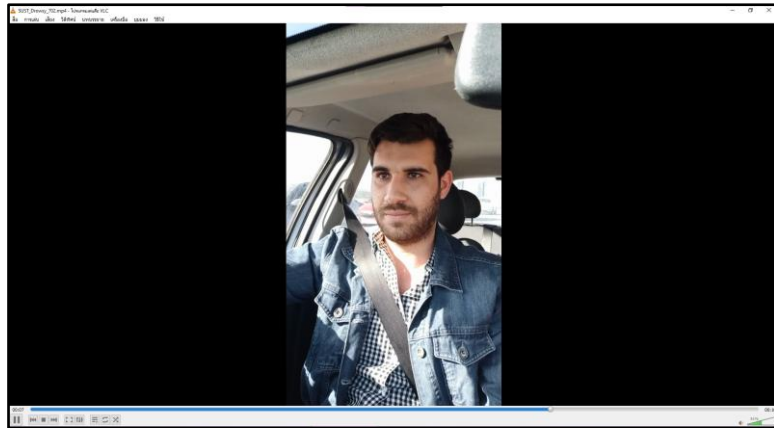
รูปที่ 3.4 ตัวอย่างวิดีโอจาก UTA-RLDD

2) Yawning Detection Dataset (YawDD) ฐานข้อมูลนี้จัดทำโดย University of Ottawa มีวัตถุประสงค์หลักเพื่อใช้ตรวจจับการหาว (Yawning) ซึ่งเป็นหนึ่งในสัญญาณของความง่วง โดยบันทึกวิดีโอผู้ขับขี่ในหลายมุม และหลายสภาพแสง ทำให้ได้ข้อมูลลักษณะการอ้าปาก การหลับตา และการแสดงออกทางสีหน้าที่เกี่ยวข้องกับอาการง่วง ดังรูปที่ 3.5



รูปที่ 3.5 ตัวอย่างวิดีโอจาก YawDD

3) SUST Driver Drowsiness Detection Dataset (SUST-DDD) เป็นชุดข้อมูลที่เผยแพร่ผ่านเว็บไซต์ Kaggle โดยมหาวิทยาลัย SUST ประเทศตุรกี ซึ่งรวบรวมวิดีโอผู้ขับขี่หลายรายภายใต้สภาวะต่าง ๆ เช่น การขับกลางวัน กลางคืน และสภาพแสงที่เปลี่ยนแปลง ชุดข้อมูลนี้แบ่งคลาสอย่างชัดเจนเป็น Awake และ Drowsy จึงเหมาะสำหรับการทดสอบความสามารถของโมเดลในสภาวะหลากหลาย ดังรูปที่ 3.6



รูปที่ 3.6 ตัวอย่างวิดีโอจาก SUST-DDD

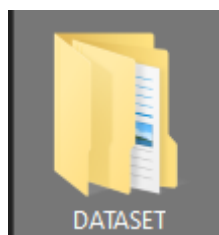
3.3.3 จำนวนตัวอย่าง

ข้อมูลทั้งหมดที่ใช้ในการฝึกสอนโมเดลประกอบด้วยวิดีโอจากการถ่ายทำด้วยตนเอง และจากฐานข้อมูลสาธารณะ โดยหลังจากการคัดกรอง และทำความสะอาดข้อมูลแล้ว ได้จำนวนคลิปวิดีโอรวมทั้งสิ้น 2,559 ไฟล์ ซึ่งถูกปรับอัตราเฟรมเรตให้เหลือ 5 เฟรมต่อวินาที เพื่อให้เหมาะสมกับการประมวลผล และการสร้างลำดับภาพ (Sequence) ความยาว 50 เฟรมสำหรับใช้เป็นข้อมูลนำเข้าโมเดล LSTM

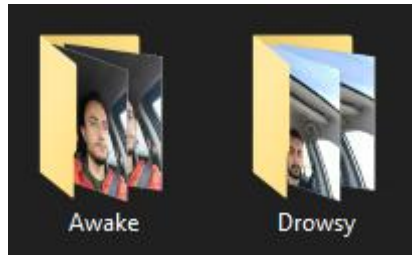
3.3.4 วิธีจัดเก็บไฟล์ และฐานข้อมูล

เพื่อให้กระบวนการฝึกสอน และทดสอบโมเดลปัญญาประดิษฐ์สำหรับการตรวจจับอาการหลับในสามารถดำเนินการได้อย่างมีประสิทธิภาพ ข้อมูลที่ได้จากการถ่ายทำ และจากฐานข้อมูลสาธารณะทั้งหมดจะถูกจัดเก็บ และจัดระเบียบให้อยู่ในรูปแบบมาตรฐาน เพื่อให้ง่ายต่อการเข้าถึง และการประมวลผลในขั้นตอนต่อไป

ข้อมูลทั้งหมดถูกจัดเก็บไว้ในโฟลเดอร์หลักชื่อ DATASET ซึ่งภายในประกอบด้วยคลาสน้อยตามระดับอาการของผู้ขับขี่ ได้แก่ Awake และ Drowsy เพื่อให้สามารถระบุประเภทของพฤติกรรมได้อย่างชัดเจน จากนั้นจึงนำข้อมูลทั้งหมดมาผ่านกระบวนการแบ่งชุดข้อมูล (Split Data) ออกเป็น 3 ส่วน ได้แก่ DATASET TRAIN, DATASET VAL และ DATASET TEST เพื่อใช้ในการฝึกสอน การตรวจสอบความถูกต้อง และการทดสอบโมเดลตามลำดับ ดังรูปที่ 3.7 และรูปที่ 3.8



รูปที่ 3.7 โฟลเดอร์ DATASET



รูปที่ 3.8 ภายในโฟลเดอร์ DATASET

3.4 การออกแบบอัลกอริทึม

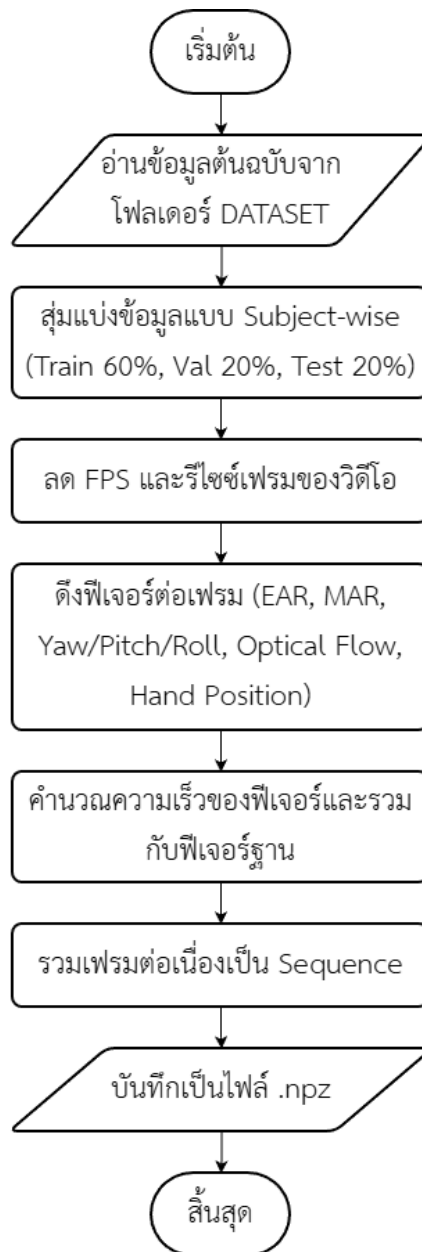
ส่วนของการออกแบบของปัญญาประดิษฐ์สำหรับระบบแจ้งเตือนการหลับในของผู้ขับขี่ยานพาหนะมีดังนี้

3.4.1 โครงสร้างโมเดล AI

การออกแบบอัลกอริทึมในโครงงานนี้มุ่งเน้นการประยุกต์ใช้ปัญญาประดิษฐ์สำหรับการตรวจจับการหลับในของผู้ขับขี่จากภาพวิดีโอแบบเรียลไทม์ให้สามารถทำงานแบบ อัตโนมัติ ตั้งแต่การตรวจจับใบหน้า และอวัยวะสำคัญ เช่น ดวงตา ปาก และศีรษะ ไปจนถึงการวิเคราะห์ลักษณะทางกายภาพเชิงเวลา เช่น การกระพริบตา การหายใจ และการพองศีรษะ พร้อมทั้งประเมินระดับความง่วงของผู้ขับขี่อย่างเป็นระบบ อัลกอริทึมถูกพัฒนาโดยใช้ภาษาไพทอน (Python) ทำงานในโปรแกรมวิชวลสตูดิโอโค้ด (Visual Studio Code)

3.4.2 การเตรียม Dataset

ทำการแบ่งชุดข้อมูลแบบ Subject-wise เพื่อป้องกันไม่ให้ข้อมูลของบุคคลเดียวกันปรากฏในหลายชุด โดยกำหนดอัตราส่วนของชุด Train : Validation : Test เท่ากับ 60 : 20 : 20 จากนั้นอ่านวิดีโอจากแต่ละชุดข้อมูล และลดอัตราเฟรมลงเหลือ 5 เฟรมต่อวินาที เพื่อเพิ่มประสิทธิภาพในการประมวลผลเฟรมจากวิดีโอถูกตรวจจับใบหน้า ดวงตา ปาก และมือ ด้วยเทคนิค MediaPipe, FaceMesh และ MediaPipe Hands ร่วมกับ Haar Cascade ในบางกรณี เพื่อคำนวณค่าพีเจอร์ เช่น EAR, MAR, มุมการเอียงของศีรษะ (Yaw, Pitch, Roll), การเคลื่อนไหวของภาพ (Optical Flow) และตำแหน่งใบหน้า (Face Position) จากนั้นนำเฟรมต่อเนื่องมารวมเป็นลำดับเวลา (Sequence) ความยาว 50 เฟรม เพื่อใช้เป็นข้อมูลนำเข้าสำหรับการเรียนรู้ของโมเดล LSTM ดังรูปที่ 3.9



รูปที่ 3.9 โฟลวชาร์ตการเตรียม Dataset

```

import os, re, time, glob, math, random, shutil, hashlib, json
from collections import defaultdict
from typing import Optional, Tuple, List
import numpy as np
import numpy as _np
from numpy.linalg import norm as _norm
import cv2
import mediapipe as mp
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers, regularizers
from tensorflow.keras import mixed_precision
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_fscore_support
from sklearn.linear_model import LogisticRegression
from sklearn.utils.class_weight import compute_class_weight

```

รูปที่ 3.10 คำสั่งเตรียมสภาพแวดล้อม และติดตั้งไลบรารีที่จำเป็น

จากรูปที่ 3.10 เป็นคำสั่งสำหรับ Import ไลบรารี ที่ใช้ในการทำงานของระบบ โดยนำเข้าโมดูลสำคัญ เช่น NumPy สำหรับการคำนวณทางคณิตศาสตร์, OpenCV และ Mediapipe สำหรับประมวลผลภาพ, และ TensorFlow สำหรับสร้าง และเทรนโมเดลปัญญาประดิษฐ์ รวมถึง Sklearn เพื่อคำนวณค่าประสิทธิภาพของโมเดล

```
DATASET_ROOT = "DATASET"
CLASS_NAMES = ["Awake", "Drowsy"]

def list_videos(dataset_root: str, class_names: List[str]) -> List[Tuple[str, int]]:
    pairs = []
    for ci, cname in enumerate(class_names):
        class_dir = os.path.join(dataset_root, cname)
        if not os.path.isdir(class_dir):
            print(f"[WARN] Missing dir: {class_dir}"); continue
        for ext in ("*.mp4", "*.mov", "*.avi", "*.mkv", "*.m4v", "*.mpg", "*.mpeg"):
            pairs.extend([(p, ci) for p in glob.glob(os.path.join(class_dir, ext))])
    return pairs
```

รูปที่ 3.11 คำสั่งอ่านข้อมูลวิดีโอต้นฉบับจากโฟลเดอร์ DATASET

จากรูปที่ 3.11 เป็นคำสั่งที่ใช้อ่านข้อมูลวิดีโอต้นฉบับจากโฟลเดอร์ DATASET โดยโปรแกรมจะอ่านรายชื่อไฟล์วิดีโอทั้งหมดจากแต่ละคลาส (Awake / Drowsy) และสร้างรายการ (Video Path, Label) เพื่อใช้ในการประมวลผลลำดับถัดไป

```
SPLITS = {"train": 0.60, "val": 0.20, "test": 0.20}

def _split_subjects(subjects):
    sids = [sid for sid in subjects.keys() if any(subjects[sid].values())]
    random.shuffle(sids)
    n = len(sids)
    n_train = int(n * SPLITS["train"]); n_val = int(n * SPLITS["val"])
    return {"train": sids[:n_train], "val": sids[n_train:n_train+n_val], "test": sids[n_train+n_val:]}
```

รูปที่ 3.12 คำสั่งสุ่มแบ่งข้อมูลแบบ Subject-wise

จากรูปที่ 3.12 เป็นคำสั่งที่ใช้แบ่งกลุ่มข้อมูลโดยยึดตามรหัส Subject เพื่อป้องกันไม่ให้คนเดียวอยู่ในหลายชุด ผลลัพธ์คือ Train, Validation และ Test ตามอัตราส่วนที่กำหนด

```

TARGET_FPS = 5
FRAME_SIZE = 160

def downsampled_frames(cap: cv2.VideoCapture, target_fps: int):
    orig_fps = cap.get(cv2.CAP_PROP_FPS)
    if orig_fps <= 0 or math.isnan(orig_fps): orig_fps = 30.0
    step = max(1, int(round(orig_fps / target_fps)))
    frames = []; idx = 0
    while True:
        ret, f = cap.read()
        if not ret: break
        if idx % step == 0: frames.append(f)
        idx += 1
    return int(round(orig_fps)), frames

```

รูปที่ 3.13 คำสั่งลด FPS และ Resize เฟรมของวิดีโอ

จากรูปที่ 3.13 เป็นคำสั่งที่ใช้ปรับอัตราเฟรมเรตวิดีโอให้เหลือ 5 เฟรมต่อวินาที (Frames Per Second: FPS) และรีไซส์ (Resize) ภาพทุกเฟรมให้มีขนาด 160×160 พิกเซล เพื่อประหยัดเวลา และหน่วยความจำในการประมวลผล

```

def frame_features(prev_frame_gray, frame_bgr, face_mesh, hands):
    face_res = face_mesh.process(rgb)
    hand_res = hands.process(rgb)
    EAR_l = eye_aspect_ratio(lm, [33,160,158,133,153,144])
    EAR_r = eye_aspect_ratio(lm, [362,385,387,263,373,380])
    MAR = mouth_aspect_ratio(lm)
    yaw,pitch,roll = head_pose(frame, lm)
    flow_mag = compute_optical_flow_mag(p0, p1)

```

รูปที่ 3.14 คำสั่งดึงฟีเจอร์ต่อเฟรม

จากรูปที่ 3.14 เป็นคำสั่งที่ใช้ MediaPipe, FaceMesh และ Hands คำนวณคุณลักษณะสำคัญของดวงตา ปาก ศีรษะ และมือ รวมถึงค่าการเคลื่อนไหวของภาพ (Optical Flow) สำหรับแต่ละเฟรม

```

FEAT_DIM = FEAT_DIM_ORIGINAL * 2

if not hasattr(frame_features, 'prev_feat'):
    frame_features.prev_feat = feat.copy()
    feat_velocity = np.zeros_like(feat)
else:
    feat_velocity = feat - frame_features.prev_feat
    feat_velocity = np.clip(feat_velocity, -1.0, 1.0)
    frame_features.prev_feat = feat.copy()

feat_combined = np.concatenate([feat, feat_velocity], axis=0).astype(np.float32)
dbg = {"gray": gray}
return feat_combined, dbg

```

รูปที่ 3.15 คำสั่งคำนวณความเร็วของฟีเจอร์ และรวมกับฟีเจอร์ฐาน

จากรูปที่ 3.15 เป็นคำสั่งที่ใช้สร้างฟีเจอร์เชิงเวลาโดยคำนวณความเปลี่ยนแปลงของค่าฟีเจอร์ระหว่างเฟรม แล้วรวมเข้ากับฟีเจอร์เดิม ทำให้โมเดลเข้าใจการเคลื่อนไหวต่อเนื่องได้ดีขึ้น

```

SEQUENCE_LEN = 50
stride = max(1, int(STRIDE_FRAMES))
seq_starts = list(range(0, max(0, len(feats)-SEQUENCE_LEN+1), stride))
if FORCE_LAST_WINDOW:
    last_start = max(0, len(feats)-SEQUENCE_LEN)
    if not seq_starts or seq_starts[-1] != last_start:
        seq_starts.append(last_start)
if not seq_starts:
    X_empty = np.empty((0, SEQUENCE_LEN, FEAT_DIM), dtype=np.float32)
    y_empty = np.empty((0,), dtype=np.int64)
    np.savez_compressed(cache_path, X=X_empty, y=y_empty)
    return X_empty, y_empty
sequences = np.stack([feats[s:s+SEQUENCE_LEN] for s in seq_starts], axis=0)

```

รูปที่ 3.16 คำสั่งรวมเฟรมต่อเนื่องเป็นลำดับเวลา (Sequence)

จากรูปที่ 3.16 เป็นคำสั่งที่ใช้รวมเฟรมต่อเนื่อง 50 เฟรมต่อหนึ่งลำดับ (Sequence) โดยเลื่อนหน้าต่างตาม Stride ที่กำหนด เพื่อสร้างข้อมูลนำเข้าสำหรับโมเดล LSTM

```

np.savez_compressed(cache_path, X=sequences, y=labels)
out_npz = f"features_{name.lower()}.npz"
np.savez_compressed(out_npz, X=X, y=y)

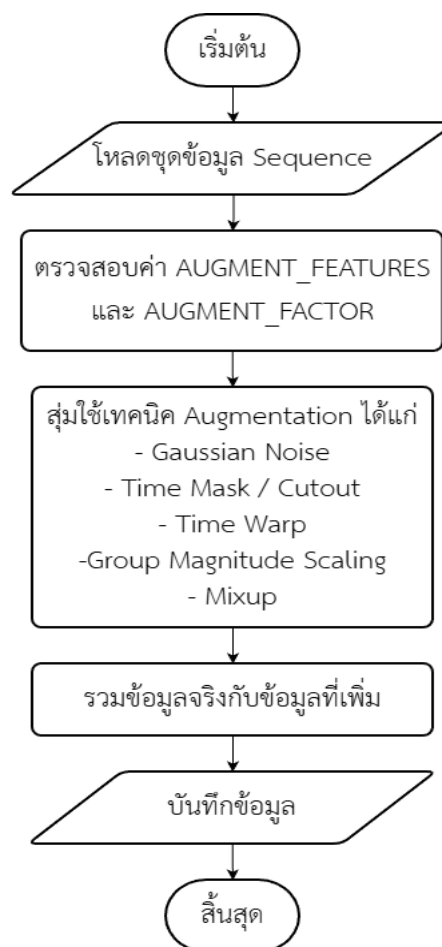
```

รูปที่ 3.17 คำสั่งบันทึกเป็นไฟล์

จากรูปที่ 3.17 เป็นคำสั่งที่ใช้บันทึกข้อมูลฟีเจอร์ที่ประมวลผลแล้วเป็นไฟล์ .npz ซึ่งเก็บทั้ง Feature และ Label ไว้ใช้ในการฝึกสอน และทดสอบโมเดลภายหลัง

3.4.3 การเพิ่มข้อมูล

ทำการเพิ่มข้อมูลเชิงเวลา (Data Augmentation) บนลำดับฟีเจอร์ด้วยเทคนิคที่สุ่มใช้ได้แก่ Gaussian Noise, Time Mask/Cutout, Time Warp, Time Shift, Segment Permutation, Group Magnitude Scaling และ Mixup ภายในคลาสเดียวกัน เพื่อเพิ่มความหลากหลายของพฤติกรรม และลดการเกิดโอเวอร์ฟิตติ้ง (Overfitting) ระหว่างการฝึกสอนโมเดล ดังรูปที่ 3.18



รูปที่ 3.18 โฟลวชาร์ตการทำ Data Augmentation

```
AUGMENT_FEATURES = True  
AUGMENT_FACTOR = 0.3
```

รูปที่ 3.19 คำสั่งตั้งค่าการทำ Data Augmentation

จากรูปที่ 3.19 เป็นคำสั่งที่ใช้กำหนดหลักในการเปิดหรือปิดโหมดการเพิ่มข้อมูล และระบุสัดส่วนของข้อมูลที่จะถูกขยายโดยเพิ่มขึ้นราว 30% ของข้อมูลเดิม

```
def augment_sequences_custom(seqs: np.ndarray, factor: float, labels: Optional[np.ndarray]=None) -> np.ndarray:
    if factor <= 0 or seqs.size == 0:
        return np.empty((0,)+seqs.shape[1:], dtype=seqs.dtype)
    N, T, D = seqs.shape
    n_out = max(1, int(round(N*factor)))
    out = []
    for _ in range(n_out):
        i = np.random.randint(0, N)
        s = seqs[i].astype(np.float32).copy()
        if AUG_NOISE_STD > 0:
            s += np.random.normal(0.0, AUG_NOISE_STD, size=s.shape).astype(np.float32)
        if AUG_TIME_MASK_PROB > 0:
            mask = (np.random.rand(T) < AUG_TIME_MASK_PROB)
            s[mask, :] = 0.0
        if np.random.rand() < AUG_YAW_FLIP_PROB and "yaw" in FEATURE_NAMES:
            yaw_idx = FEATURE_NAMES.index("yaw")
            s[:, yaw_idx] = -s[:, yaw_idx]
            s[:, yaw_idx + FEAT_DIM_ORIGINAL] = -s[:, yaw_idx + FEAT_DIM_ORIGINAL]
        if np.random.rand() < AUG_TIME_WARP_PROB:
            s = _time_warp_resample(s, *AUG_TIME_WARP_RANGE)
        s = _time_shift(s, AUG_TIME_SHIFT_FRAMES)
        if AUG_CUTOUT_MAX_T > 0 and np.random.rand() < 0.60:
            s = _temporal_cutout(s, AUG_CUTOUT_MAX_T)
        s = _group_scale(s, GROUPS, *AUG_MAG_SCALE_RANGE)
        if np.random.rand() < AUG_SEG_PERMUTE_PROB:
            s = _permute_segments(s, n_seg=AUG_SEGMENTS)
        if labels is not None and np.random.rand() < AUG_MIXUP_PROB:
            same = np.arange(N) if len(labels)==0 else np.where(labels == labels[i])[0]
            if len(same) > 1:
                j = int(np.random.choice(same[same!=i]))
                s2 = seqs[j].astype(np.float32)
                s = _mixup_pair(s, s2, AUG_MIXUP_ALPHA)
        out.append(s[None, ...])
    return np.concatenate(out, axis=0).astype(np.float32)
```

รูปที่ 3.20 คำสั่งสุมใช้เทคนิค Augmentation

จากรูปที่ 3.20 เป็นคำสั่งใช้สุมเลือกเทคนิคการเพิ่มข้อมูล เช่น

- Gaussian Noise ใส่สัญญาณรบกวนเล็กน้อย
- Time Mask / Cutout ปิดบางช่วงของลำดับเวลา (Sequence)
- Time Warp ปรับความเร็วของลำดับเวลา (Sequence) ให้เร็วหรือช้าลง
- Time Shift เลื่อนลำดับเวลา (Sequence) ทั้งชุดไปข้างหน้าหรือถอยหลัง
- Group Magnitude Scaling ปรับสเกลของกลุ่มฟีเจอร์ที่เกี่ยวข้องกัน
- Segment Permutation แบ่งลำดับออกเป็นช่วง และสลับลำดับแบบสุ่ม
- Mixup ผสมลำดับจากคลาสเดียวกันสองตัวเพื่อสร้างข้อมูลใหม่

เทคนิคเหล่านี้ช่วยเพิ่มความหลากหลายให้กับพฤติกรรมของผู้ขับที่ เช่น ความเร็วของการ กระพริบตา การหายใจ หรือการเอียงศีรษะ ทำให้โมเดลมีความสามารถในการเรียนรู้ที่ยืดหยุ่นขึ้น และลดการ เกิดโอเวอร์ฟิตติ้ง (Overfitting) ได้อย่างมีประสิทธิภาพ

```
out.append(s[None, ...])
return np.concatenate(out, axis=0).astype(np.float32)
np.savez_compressed(cache_path, X=sequences, y=labels)
```

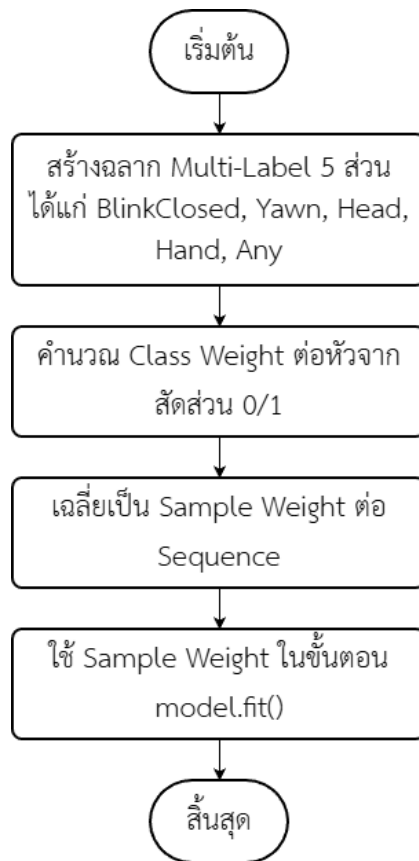
รูปที่ 3.21 คำสั่งรวมข้อมูลจริงกับข้อมูลที่เพิ่ม และบันทึกข้อมูล

จากรูปที่ 3.21 เป็นคำสั่งที่ใช้รวมลำดับเวลา (Sequence) เดิมกับลำดับเวลา (Sequence) ที่ผ่านการเพิ่มข้อมูลแล้ว เพื่อให้ชุดฝึกสอนมีความหลากหลาย และขนาดใหญ่ขึ้นก่อนเทรนโมเดล และทำการบันทึกชุดข้อมูลหลังการเพิ่มข้อมูลเป็นไฟล์ .npz เพื่อใช้ในการฝึกสอนโมเดลต่อไปอย่างมีประสิทธิภาพ และลดเวลาในการโหลดซ้ำ

3.4.4 การทำให้ข้อมูลสมดุล (Data Balancing)

การทำให้ข้อมูลมีความสมดุล (Data Balancing) ระบบได้ดำเนินการสร้างกระบวนการถ่วงน้ำหนักของข้อมูลเพื่อให้การเรียนรู้ของโมเดลมีความยุติธรรม และลดความลำเอียงที่อาจเกิดจากจำนวนตัวอย่างที่ไม่เท่ากันในแต่ละคลาส โดยเริ่มจากการสร้างฉลากแบบหลายป้ายกำกับ (Multi-Label) สำหรับการตรวจจับทั้ง 5 หัวข้อ ได้แก่ BlinkClosed, Yawn, Head, Hand และ Any ซึ่งแต่ละหัวข้อจะมีการประเมินสัดส่วนของค่าที่เป็น 0 และ 1 เพื่อใช้ในการคำนวณค่าน้ำหนักของแต่ละคลาส (Class Weight) จากนั้นระบบจะเฉลี่ยค่าน้ำหนักในแต่ละหัวเพื่อแปลงเป็นค่าน้ำหนักระดับตัวอย่าง (Sample Weight) ประจำแต่ละลำดับเวลา (Sequence) เพื่อใช้ในการฝึกโมเดลในลำดับต่อไป ค่าถ่วงน้ำหนักนี้ช่วยให้ข้อมูลที่พบได้น้อยมีอิทธิพลต่อการเรียนรู้มากขึ้น และลดการลำเอียงของโมเดลต่อพฤติกรรมที่เกิดขึ้นบ่อยเกินไป

ในขั้นตอนการฝึกสอน โมเดลจะนำค่า Sample Weight ที่คำนวณได้ไปใช้ในฟังก์ชัน model.fit() เพื่อให้กระบวนการเรียนรู้สามารถปรับอัตราการอัปเดตน้ำหนักตามความสมดุลของข้อมูลในแต่ละหัวตรวจจับได้โดยอัตโนมัติ ส่งผลให้โมเดลสามารถเรียนรู้ลักษณะของพฤติกรรมทั้ง 5 รูปแบบได้อย่างเท่าเทียม และแม่นยำมากยิ่งขึ้น การออกแบบกระบวนการนี้ยังช่วยลดปัญหาความไม่สมดุลของข้อมูลโดยไม่จำเป็นต้องเพิ่มหรือลดจำนวนตัวอย่างจริง ทำให้ได้ชุดข้อมูลที่คงความเป็นธรรมชาติ และมีประสิทธิภาพในการฝึกโมเดล ดังรูปที่ 3.22



รูปที่ 3.22 โฟลวชาร์ตการทำ Data Balancing

```
def make_multilabel_targets(X_with_temporal: np.ndarray) -> np.ndarray:
    y_mt = np.zeros((X_with_temporal.shape[0], NUM_TASKS), dtype=np.int32)
    for i in range(X_with_temporal.shape[0]):
        y_mt[i] = _seq_rule_labels(X_with_temporal[i])
    return y_mt
```

รูปที่ 3.23 คำสั่งสร้างฉลาก Multi-Label 5 ส่วน

จากรูปที่ 3.23 เป็นคำสั่งที่ใช้สร้างฉลากหลายหัว (Multi-Label) จำนวน 5 ส่วน คือ BlinkClosed, Yawn, Head, Hand และ Any โดยใช้ฟังก์ชัน make_multilabel_targets() ซึ่งอ้างอิงจากกฎใน _seq_rule_labels() ที่ตรวจ EAR, MAR, yaw/pitch และ hand_flag เพื่อสร้างค่าป้ายกำกับ 0/1

```

class_weights = []
for k in range(NUM_TASKS):
    yk = y_train_mt[:,k]
    if yk.sum()==0 or yk.sum()==len(yk):
        class_weights.append((1.0,1.0))
    else:
        w0 = len(yk)/(2.0*(len(yk)-yk.sum()))
        w1 = len(yk)/(2.0*(yk.sum()))
        class_weights.append((w0,w1))

```

รูปที่ 3.24 คำสั่งคำนวณ Class Weight

จากรูปที่ 3.24 เป็นคำสั่งที่ใช้คำนวณน้ำหนักของแต่ละคลาส (0 และ 1) ในแต่ละหัวตามสัดส่วนความถี่ของข้อมูล เพื่อให้ตัวอย่างที่พบบได้น้อยมีน้ำหนักมากกว่าตัวอย่างที่พบบมาก ป้องกันไม่ให้โมเดลเอนเอียง

```

sw = np.zeros((len(y_train_mt),), dtype=np.float32)
for i in range(len(y_train_mt)):
    w = 0.0
    for k in range(NUM_TASKS):
        w += class_weights[k][ y_train_mt[i,k] ]
    sw[i] = w/NUM_TASKS

```

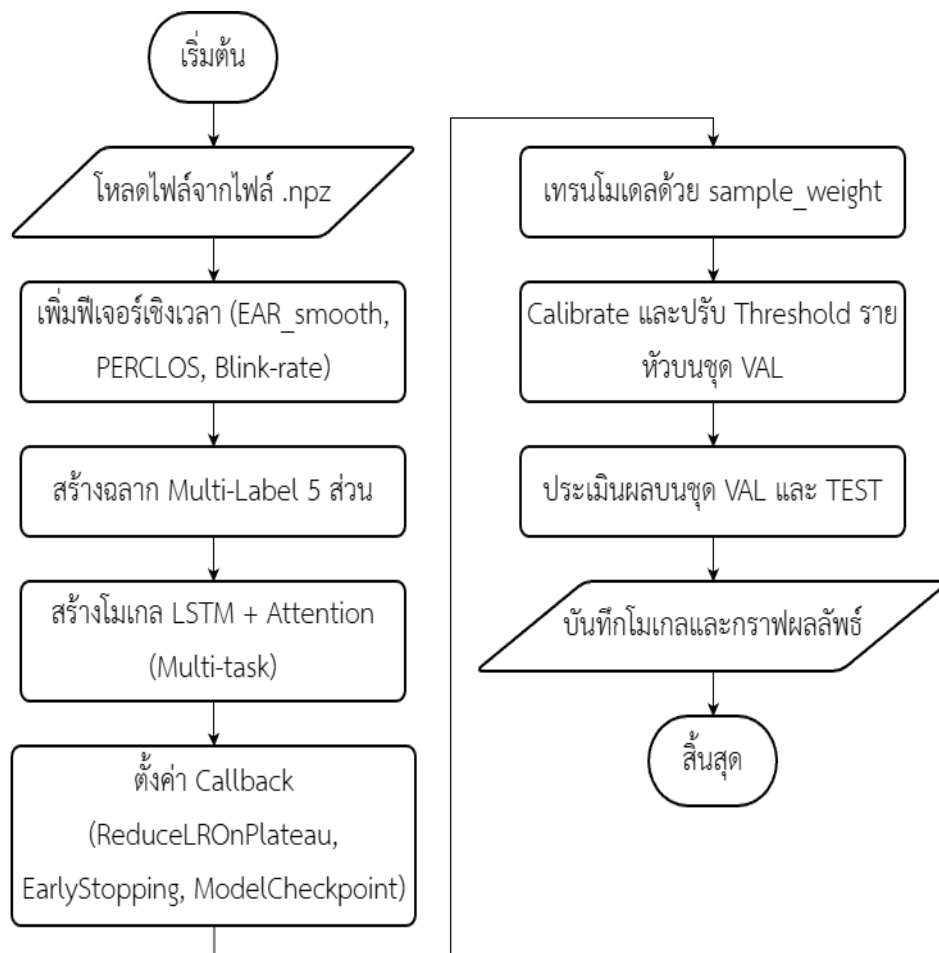
รูปที่ 3.25 คำสั่งเฉลี่ยเป็น Sample Weight ต่อลำดับเวลา (Sequence)

จากรูปที่ 3.25 เป็นคำสั่งที่ใช้เฉลี่ยค่าน้ำหนักของทั้ง 5 ส่วนต่อหนึ่งลำดับเวลา (Sequence) กลายเป็นค่า sample_weight เพื่อให้แต่ละตัวอย่างมีความสำคัญตามระดับความสมดุลโดยรวมของทุกหัวพร้อมกัน

3.4.5 การฝึกสอนให้อัลกอริทึมเกิดการเรียนรู้

โมเดลที่ใช้เป็นแบบ LSTM ร่วมกับ Attention (Multi-Task Learning) ซึ่งสามารถเรียนรู้ข้อมูลที่มีลำดับเวลา และทำการจำแนกได้หลายหัวพร้อมกัน (BlinkClosed, Yawn, Head, Hand, Any) ข้อมูลที่ใช้ในการฝึกสอนเป็นชุดที่ผ่านการเพิ่มข้อมูล และปรับสมดุลแล้ว จากนั้นเพิ่มพีเจอร์เชิงเวลาเพิ่มเติม ได้แก่ EAR_smooth, PERCLOS, และ Blink-rate เพื่อช่วยให้โมเดลเข้าใจการเปลี่ยนแปลงพฤติกรรมต่อเนื่องได้ดีขึ้น

โมเดลถูกสร้างขึ้นด้วยชั้น Bidirectional LSTM และ Multi-Head Attention ตามด้วย Dense Layer สำหรับการจำแนกทั้ง 5 หัว พร้อมเทคนิค Dropout และ Batch Normalization เพื่อป้องกันการโอเวอร์ฟิตติ้ง (Overfitting) การฝึกสอนใช้พารามิเตอร์ `sample_weight` จากขั้นตอนปรับสมดุล เพื่อชดเชยความไม่เท่ากันของคลาส และมีการตั้งค่า Callback ได้แก่ EarlyStopping, ReduceLROnPlateau และ ModelCheckpoint เพื่อเพิ่มประสิทธิภาพการเรียนรู้ และหยุดเมื่อโมเดลเริ่มโอเวอร์ฟิต (Overfit) หลังการฝึกสอน โมเดลจะถูกคาลิเบรต (Calibrate) และปรับเทรชโฮลด์ (Threshold) ของแต่ละหัวด้วยข้อมูล VAL โดยใช้ Logistic Regression (Platt scaling) เพื่อหาค่าเทรชโฮลด์ (Threshold) ที่ให้ F1-Score สูงสุด ก่อนนำไปประเมินผลบนชุด VAL และ TEST จากนั้นบันทึกโมเดลพร้อมกราฟผลลัพธ์ไว้เพื่อวิเคราะห์ประสิทธิภาพ ดังรูปที่ 3.26



รูปที่ 3.26 โฟลวชาร์ตการฝึกสอนให้อัลกอริทึมเกิดการเรียนรู้

```
def add_temporal_features(X, ear_thr=0.20, win_smooth=7, win_perclos=17):
    ear_idx = FEATURE_NAMES.index("EAR")
    ear = X[:, :, ear_idx]
    kernel = np.ones(win_smooth)/win_smooth
    ear_smooth = np.apply_along_axis(lambda v: np.convolve(v, kernel, mode="same"), 1, ear)
    closed = (ear < ear_thr).astype(float)
    k2 = np.ones(win_perclos)/win_perclos
    perclos = np.apply_along_axis(lambda v: np.convolve(v, k2, mode="same"), 1, closed)
    blink_edges = np.abs(np.diff(closed, axis=1, prepend=closed[:, :1]))
    blink_rate = np.apply_along_axis(lambda v: np.convolve(v, k2, mode="same"), 1, blink_edges)
    return np.concatenate([X, ear_smooth[... ,None], perclos[... ,None], blink_rate[... ,None]], axis=2)
```

รูปที่ 3.27 คำสั่งเพิ่มฟีเจอร์เชิงเวลา

จากรูปที่ 3.27 เป็นคำสั่งที่ใช้เพิ่มฟีเจอร์ต่อเวลา เช่น EAR_smooth, PERCLOS, และ Blink-rate ซึ่งได้จากการคำนวณค่าเฉลี่ย และอัตราการกะพริบของตา เพื่อช่วยให้โมเดลจับรูปแบบการง่วงนอนได้ดีขึ้น

```
def build_lstm_multitask(input_timesteps: int, input_dim: int, num_tasks: int):
    inputs = layers.Input(shape=(input_timesteps, input_dim))
    x = layers.Masking(mask_value=0.0)(inputs)
    x = layers.SpatialDropout1D(0.15)(x)
    x = layers.GaussianNoise(0.010)(x)
    x = layers.Conv1D(64, 5, padding="same", activation="relu",
                      kernel_regularizer=regularizers.l2(4e-4))(x)
    x = layers.BatchNormalization()(x)
    x = layers.SpatialDropout1D(0.10)(x)
    x = layers.Bidirectional(layers.LSTM(
        40, return_sequences=True,
        recurrent_dropout=0.35,
        kernel_regularizer=regularizers.l2(4e-4)
    ))(x)
    x = layers.LayerNormalization()(x)
    attn = layers.MultiHeadAttention(num_heads=4, key_dim=32, dropout=0.25)(x, x)
    x = layers.Add()([x, attn])
    x = layers.GlobalAveragePooling1D()(x)
    x = layers.Dense(64, activation="relu", kernel_regularizer=regularizers.l2(4e-4))(x)
    x = layers.Dropout(0.55)(x)
    outputs = layers.Dense(num_tasks, activation="sigmoid", dtype="float32")(x)
    model = models.Model(inputs, outputs)
    model.compile(
        optimizer=optimizers.Adam(8e-5),
        loss=tf.keras.losses.BinaryCrossentropy(),
        metrics=[tf.keras.metrics.BinaryAccuracy(name="bin_acc")]
    )
    return model
```

รูปที่ 3.28 คำสั่งสร้างโมเดล LSTM + Attention (Multi-Task)

จากรูปที่ 3.28 เป็นคำสั่งที่ใช้สร้างโมเดล LSTM หลายหัวพร้อมชั้น Attention เพื่อให้เรียนรู้บริบทของข้อมูลเชิงเวลาในแต่ละลำดับเวลา (Sequence) และแบ่งปันการเรียนรู้ระหว่างส่วนได้มีประสิทธิภาพ

```
callbacks = [
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss" if has_val else "loss",
                                          factor=0.5, patience=2, min_lr=1e-6, verbose=1),
    tf.keras.callbacks.EarlyStopping(monitor="val_loss" if has_val else "loss",
                                     patience=6, min_delta=1e-4, restore_best_weights=True, verbose=1),
    tf.keras.callbacks.ModelCheckpoint("best_lstm_multitask.keras", monitor="val_loss" if has_val else "loss",
                                       save_best_only=True, verbose=1),
    tf.keras.callbacks.TerminateOnNaN()
]
```

รูปที่ 3.29 คำสั่งตั้งค่า Callback

จากรูปที่ 3.29 เป็นคำสั่งที่ใช้ตั้งค่า Callback โดย ReduceLROnPlateau ใช้สำหรับปรับอัตราการเรียนรู้เมื่อค่า Loss หยุดลด, EarlyStopping ใช้สำหรับหยุดการเทรนอัตโนมัติเมื่อเริ่มโอเวอร์ฟิต (Overfit) และ ModelCheckpoint ใช้บันทึกโมเดลที่ดีที่สุดไว้ใช้ประเมินผลภายหลัง

```
history = model.fit(
    X_train, y_train_mt.astype(np.float32),
    validation_data=(X_val, y_val_mt.astype(np.float32)) if has_val else None,
    epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1,
    callbacks=callbacks, sample_weight=sw
)
```

รูปที่ 3.30 คำสั่งเทรนโมเดลด้วย sample_weight

จากรูปที่ 3.30 เป็นคำสั่งสำหรับฝึกสอนโมเดลโดยใช้ sample_weight เพื่อปรับสมดุลของข้อมูลในแต่ละหัวตรวจจับให้เหมาะสมกับความถี่ของข้อมูล ค่าถ่วงน้ำหนักนี้ช่วยให้โมเดลให้ความสำคัญกับข้อมูลที่มีจำนวนน้อยมากขึ้น ลดอคติจากข้อมูลที่เกิดบ่อยเกินไป และทำให้การเรียนรู้ของโมเดลมีความยุติธรรม และแม่นยำยิ่งขึ้น

```

def tune_thresholds_on_val(y_true_val, y_prob_val):
    thresholds = np.full((NUM_TASKS,), 0.5, dtype=np.float32)
    calibrators = []
    for k in range(NUM_TASKS):
        y_true_k = y_true_val[:, k]
        y_prob_k = y_prob_val[:, k]
        uniq = np.unique(y_true_k)
        if uniq.size < 2:
            calibrators.append(None)
            if uniq[0] == 1:
                th = float(np.percentile(y_prob_k, 30))
            else:
                th = float(np.percentile(y_prob_k, 70))
            thresholds[k] = np.clip(th, 0.01, 0.99)
            print(f"[WARN] Head {k}: VAL has single class {int(uniq[0])}; "
                  f"skip calibration and use th={thresholds[k]:.3f}.")
            continue
        lr = LogisticRegression(max_iter=1000)
        lr.fit(y_prob_k.reshape(-1, 1), y_true_k)
        calibrators.append(lr)
        p_cal = lr.predict_proba(y_prob_k.reshape(-1, 1))[:, 1]
        best, best_th = -1.0, 0.50
        for th in np.linspace(0.30, 0.85, 112):
            y_tmp = (p_cal >= th).astype(int)
            _, _, f1, _ = precision_recall_fscore_support(
                y_true_k, y_tmp, average=None, zero_division=0
            )
            score = f1.mean()
            if score > best:
                best, best_th = score, th
        thresholds[k] = float(best_th)
        print(f"[THRESH] {TASK_NAMES[k]} -> {thresholds[k]:.3f}")
    return thresholds, calibrators

```

รูปที่ 3.31 คำสั่งคาลิเบรต (Calibrate) และปรับเทรชโฮลด์ (Threshold) รายหัวบนชุด VAL

จากรูปที่ 3.31 เป็นคำสั่งที่ใช้ทำการคาลิเบรต (Calibrate) และหาค่าเทรชโฮลด์ (Threshold) สำหรับแต่ละหัวตรวจจับ โดยใช้วิธี Logistic Regression เพื่อปรับความน่าจะเป็นของผลลัพธ์ให้อยู่ในช่วงที่เหมาะสม จากนั้นระบบจะทำการกวาดค่าของเทรชโฮลด์ (Threshold) ในช่วง 0.30–0.85 เพื่อหาค่าที่ให้ค่า F1-Score เฉลี่ยสูงสุดบนชุดข้อมูล Validation ขั้นตอนนี้ช่วยให้แต่ละหัวตรวจจับมีเกณฑ์การตัดสินใจที่เหมาะสมกับลักษณะข้อมูลจริง ลดความผิดพลาดในการจำแนก และเพิ่มความแม่นยำของผลลัพธ์โดยรวมของโมเดล

```
def evaluate_per_head(y_true, y_prob, thresholds, split_name, out_dir):
    os.makedirs(out_dir, exist_ok=True)
    head_class_names = ["No", "Yes"]
    y_pred = (y_prob >= thresholds[None, :]).astype(int)
    table = []
    for k, name in enumerate(TASK_NAMES):
        cm = confusion_matrix(y_true[:,k], y_pred[:,k], labels=[0,1])
        plot_cm(cm, head_class_names, f"{split_name} CM - {name}", os.path.join(out_dir, f"cm_{split_name.lower()}_{name}.png"))
        p, r, f1 = per_head_metrics(y_true[:,k], y_pred[:,k])
        table.append((name, p[1], r[1], f1[1], cm.tolist()))
    f1s = [row[3] for row in table]
    plt.figure(figsize=(8,3.2))
    x = np.arange(NUM_TASKS); plt.bar(x, f1s)
    plt.xticks(x, TASK_NAMES, rotation=10)
    plt.ylim(0,1.0); plt.ylabel("F1 (class=Yes)")
    plt.title(f"{split_name}: F1 by Head")
    plt.tight_layout(); plt.savefig(os.path.join(out_dir, f"{split_name.lower()}_f1_by_head.png"), dpi=150); plt.close()
    print(f"\n[{split_name}] Metrics per head (class=Yes)")
    for name, p1, r1, f1, cm in table:
        print(f" - {name:12s} P={p1:.3f} R={r1:.3f} F1={f1:.3f} CM={cm}")
    return y_pred
```

รูปที่ 3.32 คำสั่งประเมินผลบนชุด VAL และ TEST

จากรูปที่ 3.32 เป็นคำสั่งที่ใช้ประเมินผลผลลัพธ์แยกแต่ละหัวด้วยค่าชี้วัด Accuracy, Precision, Recall และ F1-Score พร้อมสร้างกราฟสรุปผล

```
def plot_training_curves(history, out_dir):
    np.savez(os.path.join(out_dir, "training_history_multitask.npz"), **history.history)
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1); plt.plot(history.history["loss"], label="train")
    if "val_loss" in history.history: plt.plot(history.history["val_loss"], label="val")
    plt.title("Loss"); plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.legend()
    plt.subplot(1,2,2); plt.plot(history.history["bin_acc"], label="train")
    if "val_bin_acc" in history.history: plt.plot(history.history["val_bin_acc"], label="val")
    plt.title("Binary Accuracy (mean over 5 heads)"); plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.legend()
    plt.tight_layout(); plt.savefig(os.path.join(out_dir, "training_curves_multitask.png"), dpi=150); plt.close()
```

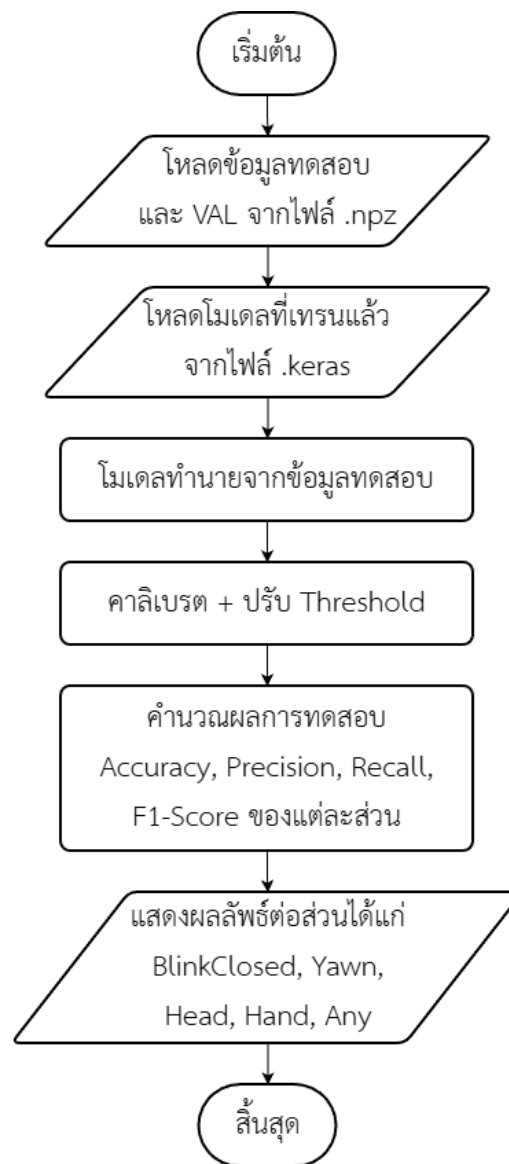
รูปที่ 3.33 คำสั่งบันทึกโมเดล และกราฟผลลัพธ์

จากรูปที่ 3.33 เป็นคำสั่งที่ใช้บันทึกโมเดลที่เทรนแล้วเป็นไฟล์ .keras และกราฟการฝึกสอน เช่น Loss และ Accuracy ในแต่ละ Epoch เพื่อใช้วิเคราะห์ประสิทธิภาพภายหลัง

3.4.6 การประเมินผลของโมเดล

ในขั้นตอนนี้ได้ทำการประเมินประสิทธิภาพของโมเดลที่ผ่านการฝึกสอนแล้ว โดยใช้ชุดข้อมูลทดสอบที่เก็บแยกไว้ไม่ซ้ำกับชุดฝึกสอน และตรวจสอบความถูกต้อง ข้อมูลถูกโหลดจากไฟล์ .npz และถูกเพิ่มฟีเจอร์เชิงเวลา เช่น EAR_smooth, PERCLOS และ Blink-rate เพื่อช่วยให้การประเมินสะท้อนพฤติกรรมจริงของผู้ขับได้มากขึ้น จากนั้นโหลดโมเดลที่ฝึกสอนแล้วจากไฟล์ .keras มาทำนายผลบนชุดทดสอบ

เริ่มจากการ คาลิเบรต (Calibrate) ผลการทำนายด้วยวิธี Logistic Regression เพื่อปรับค่าความน่าจะเป็นให้สอดคล้องกับข้อมูลจริง และหาค่าเทรชโฮลด์ (Threshold) ที่เหมาะสมที่สุดในแต่ละส่วน จากนั้นคำนวณค่าตัวชี้วัดประสิทธิภาพ ได้แก่ Accuracy, Precision, Recall และ F1-Score สำหรับแต่ละส่วนของการตรวจจับ (BlinkClosed, Yawn, Head, Hand, Any) เพื่อประเมินความสามารถของโมเดลในการแยกแยะพฤติกรรมของผู้ขับที่ถูกต้อง และรอบด้าน ดังรูปที่ 3.34



รูปที่ 3.34 โฟลวชาร์ตการประเมินผลของโมเดล

```
thresholds = np.full((NUM_TASKS,), 0.5, dtype=np.float32)
calibrators = [None]*NUM_TASKS
if y_prob_val is not None:
    thresholds, calibrators = tune_thresholds_on_val(y_val_mt, y_prob_val)
```

รูปที่ 3.35 คำสั่งคาลิเบรต (Calibrate) และปรับเทรชโฮลด์ (Threshold)

จากรูปที่ 3.35 เป็นคำสั่งที่ใช้ทำการคาลิเบรต (Calibrate) และปรับเทรชโฮลด์ (Threshold) ด้วยข้อมูล Validation เพื่อหาค่าที่ให้ F1-Score สูงสุดในแต่ละหัว ก่อนนำมาประเมินผลบนชุด Test

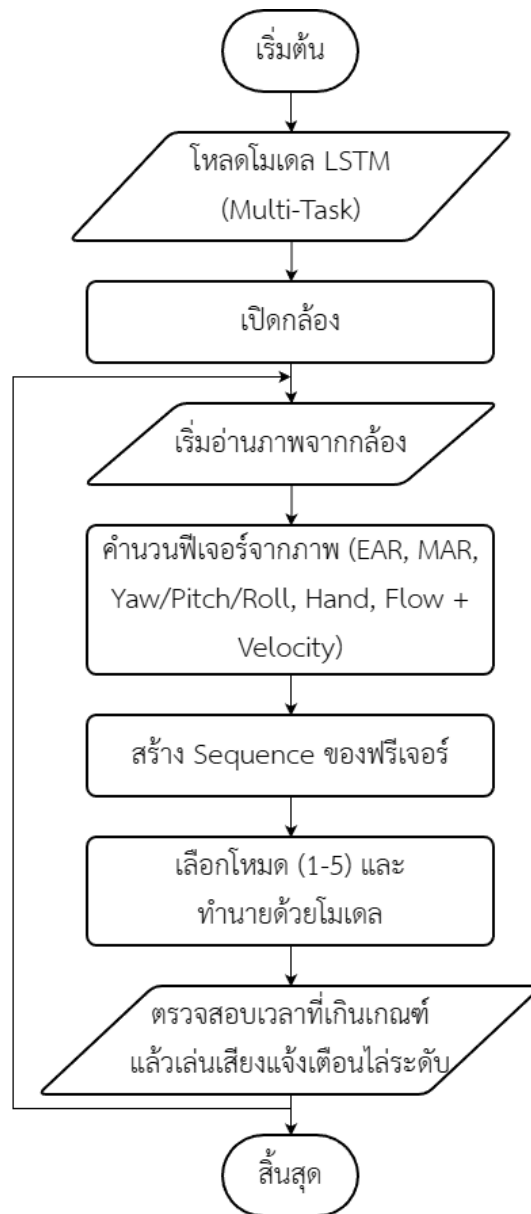
```
def print_block(y_true_bin, y_pred_bin, title):
    cm = confusion_matrix(y_true_bin, y_pred_bin, labels=[0,1])
    tn, fp, fn, tp = cm.ravel()
    acc = accuracy_score(y_true_bin, y_pred_bin)
    prec = precision_score(y_true_bin, y_pred_bin, zero_division=0)
    rec = recall_score(y_true_bin, y_pred_bin, zero_division=0)
    f1 = f1_score(y_true_bin, y_pred_bin, zero_division=0)
```

รูปที่ 3.36 คำสั่งคำนวณผลการทดสอบโมเดล

จากรูปที่ 3.36 เป็นคำสั่งที่ใช้สำหรับคำนวณผลการทดสอบโมเดล โดยจะดูว่าทำนายถูกหรือผิดจากค่า Confusion Matrix แล้วคำนวณค่า Accuracy, Precision, Recall และ F1-Score เพื่อใช้ประเมินประสิทธิภาพของโมเดล

3.4.7 การพัฒนาระบบตรวจจับอาการหลับในแบบเรียลไทม์พร้อมระบบแจ้งเตือน

ภายหลังการฝึกสอนโมเดล LSTM แบบ Multi-Task แล้ว ได้พัฒนาระบบทำงานแบบเรียลไทม์ด้วยเว็บแคม โดยในแต่ละเฟรมระบบจะปรับขนาดภาพเป็น 160×160 และสกัดพีเจอร์จาก MediaPipe FaceMesh/Hands (เช่น EAR, MAR, yaw/pitch/roll, สถานะมือใกล้ใบหน้า) พร้อมทั้งคำนวณ Optical-flow และ ความเร็วของพีเจอร์ (Velocity) ก่อนนำมารวมเป็น ลำดับเวลา (Sequence) ยาว 50 เฟรม จากนั้นปรับมาตรฐานต่อ sequence และเติมพีเจอร์เชิงเวลา (EAR_smooth, PERCLOS, Blink-rate) แล้วจึงให้โมเดลทำนายความน่าจะเป็นแยกหัวข้อ BlinkClosed, Yawn, Head, Hand, Any โดยผู้ใช้สามารถเลือกโหมดประเมิน 1-5 ได้ผลทำนายจะถูกเปรียบเทียบกับเทรชโฮลด์ (Threshold) รายหัว เพื่อจัดสถานะว่าง/ปกติ และเมื่อสถานะว่างต่อเนื่องเกินช่วงเวลาที่กำหนด ระบบจะ ใส่ระดับเสียงเตือน 4 ระดับ อัตโนมัติ เพื่อช่วยลดความเสี่ยงจากภาวะหลับในระหว่างขับขี่ ดังรูปที่ 3.37



รูปที่ 3.37 โฟลวชาร์ตการพัฒนาระบบตรวจจับอาการหลับในแบบเรียลไทม์พร้อมระบบแจ้งเตือน

```

def open_camera(target_fps=5, width=640, height=480):
    backends = []
    if hasattr(cv2, "CAP_DSHOW"): backends.append(cv2.CAP_DSHOW)
    if hasattr(cv2, "CAP_MSMF"): backends.append(cv2.CAP_MSMF)
    backends.append(None)
    for idx in [0,1,2,3]:
        for be in backends:
            cap = cv2.VideoCapture(idx) if be is None else cv2.VideoCapture(idx, be)
            if not cap.isOpened():
                cap.release(); continue
            cap.set(cv2.CAP_PROP_FPS, target_fps)
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, width)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, height)
            cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))
            ok=False
            for _ in range(5):
                ret, fr = cap.read()
                if ret: ok=True; break
                time.sleep(0.05)
            if ok:
                backend_name = "AUTO" if be is None else ("DSHOW" if be == cv2.CAP_DSHOW else "MSMF")
                print(f"[INFO] 🟢 Camera opened (backend={backend_name}, index={idx})")
                return cap
            cap.release()
    print("❌ ไม่สามารถเปิดกล้องได้")
    return None

```

รูปที่ 3.38 คำสั่งเปิดกล้อง และเริ่มอ่านภาพจากกล้อง

จากรูปที่ 3.38 เป็นคำสั่งที่ใช้เปิดกล้องด้วยแบ็คเอนด์ (Backend) แล้วตั้งค่าความละเอียดและเฟรมเรต (Frame Rate) ตาม TARGET_FPS เพื่อเริ่มการจับภาพจากเว็บแคม

```

def frame_features(prev_frame_gray, frame_bgr, face_mesh, hands):
    canvas, transform = safe_resize_pad(frame_bgr, FRAME_SIZE)
    gray = cv2.cvtColor(canvas, cv2.COLOR_BGR2GRAY)
    rgb = cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB)

    face_res = face_mesh.process(rgb)
    hand_res = hands.process(rgb)
    face, eyes = detect_face_and_eyes(gray)

    base5 = np.zeros(5, dtype=np.float32)
    fx_c = fy_c = 0.5

    if face is not None:
        (x,y,w,h) = face
        face_area = float(w*h) if w*h>0 else 1.0
        num_eyes = min(2, len(eyes))
        total_eye_area = sum(float(ew*eh) for (_,_,ew,eh) in eyes)
        flow_mag = 0.0
        if prev_frame_gray is not None:
            p0 = prev_frame_gray[y:y+h, x:x+w]
            p1 = gray[y:y+h, x:x+w]
            if p0.shape == p1.shape and p0.size>0:
                flow_mag = compute_optical_flow_mag(p0, p1)
        base5 = np.array([w/FRAME_SIZE, h/FRAME_SIZE, num_eyes/2.0,
                        (total_eye_area/face_area) if face_area>0 else 0.0,
                        flow_mag], dtype=np.float32)
        fx_c = (x + w/2) / FRAME_SIZE
        fy_c = (y + h/2) / FRAME_SIZE

```

รูปที่ 3.39 คำสั่งคำนวณฟีเจอร์จากภาพ

จากรูปที่ 3.39 เป็นคำสั่งที่ใช้สกัดพีเจอร์ 34 มิติจากภาพ เช่น EAR, MAR, ท่าศีรษะ, มือใกล้ใบหน้า และ Optical-flow แล้วเก็บค่ากลับเข้า Sequence เป็นลำดับต่อเนื่อง พร้อมบันทึก Landmark, Hand, และ Transform เพื่อใช้แสดงผล

```
X = np.expand_dims(X_full, 0)
y_probs = model.predict(X, verbose=0)
p_sel = float(y_probs[0, MODE])
mode_name = TASK_NAMES[MODE]
th = THRESHOLDS[MODE]
state = "Drowsy" if p_sel >= th else "Awake"
conf = p_sel if state == "Drowsy" else (1.0 - p_sel)
```

รูปที่ 3.40 คำสั่งเลือกโหมด และทำนายด้วยโมเดล

จากรูปที่ 3.40 เป็นคำสั่งที่ใช้เลือกผลจากโหมดที่กำหนด (BlinkClosed, Yawn, Head, Hand, หรือ Any) แล้วตัดสินใจว่าจะว่างจากเทรชโฮลด์ (Threshold) รายหัว

```
if state == "Drowsy":
    if drowsy_start is None:
        drowsy_start = time.time()
        alert_state = 0
        alert3_count = 0
        last_alert_time = 0.0
    duration = time.time() - drowsy_start
    if duration > ALERT_LEVEL_3:
        if alert3_count < ALERT3_MAX:
            if (time.time() - last_alert_time) > ALERT_INTERVAL:
                play_alert(3); alert3_count += 1
                last_alert_time = time.time(); alert_state = 3
            else:
                if (time.time() - last_alert_time) > ALERT_INTERVAL:
                    play_alert(4); last_alert_time = time.time(); alert_state = 4
        elif duration > ALERT_LEVEL_2 and alert_state < 2:
            play_alert(2); alert_state = 2; alert3_count = 0; last_alert_time = time.time()
        elif duration > ALERT_LEVEL_1 and alert_state < 1:
            play_alert(1); alert_state = 1; alert3_count = 0; last_alert_time = time.time()
    else:
        drowsy_start = None
        alert_state = 0
        alert3_count = 0
        last_alert_time = 0.0
```

รูปที่ 3.41 คำสั่งเล่นเสียงแจ้งเตือน

จากรูปที่ 3.41 เป็นคำสั่งที่ใช้เป็นระบบแจ้งเตือนอาการง่วงแบบหลายระดับ โดยทำงานเมื่อโมเดลตรวจพบ สถานะ Drowsy โปรแกรมจะเริ่มจับเวลา และตรวจสอบระยะเวลาที่อยู่ในภาวะง่วงเพื่อกำหนดระดับการแจ้งเตือน และเมื่อโมเดลตรวจพบว่าผู้ใช้กลับมามีสภาวะ Awake ระบบจะ รีเซ็ตค่าการจับเวลา และตัวนับการแจ้งเตือนทั้งหมดให้อยู่ในสถานะเริ่มต้นใหม่